



LDBC

Cooperative Project

FP7 – 317548

D2.2.2 Data Generator

Coordinator: [Irimi Fundulaki]

**With contributions from: [Irimi Fundulaki, Norbert
Martinez, Renzo Angles, Barry Bishop, Venelin Kotsev]**

1st Quality Reviewer: Orri Erling (OGL)

2nd Quality Reviewer: Alex Averbuch (NEO)

Deliverable nature:	Report (R)
Dissemination level: (Confidentiality)	Public (PU)
Contractual delivery date:	M12
Actual delivery date:	M12
Version:	1.0
Total number of pages:	47
Keywords:	Linked Open Data, RDF, Graph Databases, Data Generator

Abstract

The purpose of this deliverable is to provide a description of the data generators implemented for the *Semantic Publishing* and *Social Network* Task Forces in the context of LDBC.

EXECUTIVE SUMMARY

This is the second deliverable for WP2 *Query Processing Choke Point Analysis* which focuses on benchmarking and testing the core aspects of linked data processing. This deliverable discusses the data generators developed in the context of the two LDBC task forces, namely *Semantic Publishing Task Force* and *Social Network Task Force*.

The *Semantic Publishing Task Force* is inspired by the Media/Publishing industry and especially the BBC. This specific benchmark has business value for *media organizations* that intend to use semantic publishing for their business and for vendors of RDF data management software. The former can use the benchmark to evaluate potential RDF engines for integration into their publishing workloads. The latter will be able to use this benchmark to find limitations in their products and provide a research focus for improvement. Vendors would also be able to use the benchmark results to market their products.

The objective of the *Social Network Task Force* is to develop a benchmark for *social network* analysis. The benchmark has added value for three kinds of audiences: i) users facing *graph processing tasks* for whom the benchmark provides an excellent scenario for comparing different technologies on the basis of price and scale; ii) for database vendors who can use the benchmark to test the performance of their engines using appropriate benchmark tasks that stress the systems' performance and iii) for researchers (industrial and academic) by providing interesting challenges on different topics.

In this deliverable we focus on the *data generators* for the aforementioned benchmarks.

DOCUMENT INFORMATION

IST Project Number	FP7 – 317548	Acronym	LDBC
Full Title	LDBC		
Project URL	http://www.ldbc.eu/		
Document URL	http://www.ldbc.eu:8090/display/PROJECT/ Deliverable+summary		
EU Project Officer	Carola Carstens		

Deliverable	Number	D2.2.2	Title	Data Generator
Work Package	Number	WP2	Title	Query Processing Choke Point Analysis

Date of Delivery	Contractual	M12	Actual	M12
Status	version 1.0		final ☐	
Nature	Report (R) ☒ Prototype (P) ☐ Demonstrator (D) ☐ Other (O) ☐			
Dissemination Level	Public (PU) ☒ Restricted to group (RE) ☐ Restricted to programme (PP) ☐ Consortium (CO) ☐			

Authors (Partner)	Iriní Fundulaki (FORTH), Norbert Martinez (UPC), Renzo Angles (VU), Barry Bishop (ONTO), Venelin Kotsev (ONTO)			
Responsible Author	Name	Iriní Fundulaki	E-mail	fundul@ics.forth.gr
	Partner	FORTH	Phone	+302810391725

Abstract (for dissemination)	The purpose of this deliverable is to provide a description of the data generators implemented for the <i>Semantic Publishing</i> and <i>Social Network</i> Task Forces in the context of LDBC.
Keywords	Linked Open Data, RDF, Graph Databases, Data Generator

Version Log			
Issue Date	Rev. No.	Author	Change
20/09/2013	0.1	Iriní Fundulaki, Norbert Martinez, Renzo Angles, Barry Bishop, Venelin Kotsev	First version
30/09/2013	1.0	Iriní Fundulaki, Norbert Martinez, Renzo Angles, Barry Bishop, Venelin Kotsev	Final version

TABLE OF CONTENTS

EXECUTIVE SUMMARY	3
DOCUMENT INFORMATION	4
LIST OF FIGURES	5
LIST OF TABLES	6
1 INTRODUCTION	8
2 SEMANTIC PUBLISHING	9
2.1 Semantic Publishing: Ontologies	9
2.2 Semantic Publishing: Workloads	14
2.3 Semantic Publishing: Data Generator	16
2.4 Statistics	19
3 SOCIAL NETWORK	22
3.1 Social Network Data Generator (SNDG) Schema	22
3.2 Social Network Data Generator	25
3.2.1 Property Dictionary	25
3.2.2 Graph Generation	26
3.2.3 Implementation	27
3.3 Data Generation	28
3.4 Graph Overview	36
3.5 Degree distribution	38
3.6 Time Evolution	44
4 CONCLUSIONS	46

LIST OF FIGURES

2.1	Creative Work 0.9	10
2.2	Company Ontology 1.4	10
2.3	Core Concepts Ontology 0.6	11
2.4	CMS Ontology 1.2	12
2.5	Person Ontology 0.2	12
2.6	Provenance Ontology 1.1	13
2.7	Tagging Ontology 1.0	13
2.8	Overview Ontology 0.2	14
2.9	Distribution of tags for Creative Works	18
3.1	Social Intelligence Benchmark Data Schema	22
3.2	SNDG Entity and Relationship	23
3.3	SNDG Inheritance and Interface	23
3.4	An example of SNDG	24

LIST OF TABLES

2.1	Distribution of tags in creative works	16
2.2	<i>Creative Works</i>	19
2.3	Statistics for <i>Creative Work Type</i>	19
2.4	Statistics for <i>Creative Work Category</i>	19
2.5	Statistics for <i>Audience Type</i>	19
2.6	Statistics for <i>Primary Format</i>	20
2.7	Statistics for <i>Live Coverage</i> (count)	20
2.8	Statistics for <i>About</i> (count)	20
2.9	Statistics for <i>Mentions</i> (count)	21
2.10	Statistics for <i>Primary Content</i> (count)	21
2.11	Statistics for <i>Web Document Types</i>	21
3.1	person-[knows]->person sf=100K1Y statistics	36
3.2	Global Statistics	36
3.3	User-Knows Subgraph Statistics	36
3.4	Labels	37
3.5	Attributes	37

1 INTRODUCTION

This is the second deliverable for WP2 *Query Processing Choke Point Analysis* which focuses on benchmarking and testing the core aspects of linked data processing. This deliverable discusses the data generators developed in the context of the two LDBC task forces, namely *Semantic Publishing Task Force* and *Social Network Task Force*.

The *Semantic Publishing Task Force* is inspired by the Media/Publishing industry and especially the BBC. This specific benchmark has business value for *media organizations* that intend to use semantic publishing for their business and for vendors of RDF data management software. The former can use the benchmark to evaluate potential RDF engines for integration into their publishing workloads. The latter will be able to use this benchmark to find limitations in their products and provide a research focus for improvement. Vendors would also be able to use the benchmark results to market their products.

The objective of the *Social Network Task Force* is to develop a benchmark for *social network* analysis. The benchmark has added value for three kinds of audiences: i) users facing *graph processing tasks* for whom the benchmark provides an excellent scenario for comparing different technologies on the basis of price and scale; ii) for database vendors who can use the benchmark to test the performance of their engines using appropriate benchmark tasks that stress the systems' performance and iii) for researchers (industrial and academic) by providing interesting challenges on different topics.

In this deliverable we focus on the *data generators* for the aforementioned benchmarks. Chapter 2 discusses the data generator for the Semantic Publishing Benchmark. We present the *BBC Ontologies* used for the data generation process in Section 2.1; a short description of the *publishing* and *editorial* workloads that drive the data generation is provided in Section 2.2; Section 2.3 discusses the data generation process. Finally, in Section 2.4 we give a set of statistics for a dataset of 50M triples created by the Semantic Publishing Data Generator.

The Social Network benchmark is discussed in Chapter 3. We first discuss the schema in Section 3.1 that is used to represent the structure of a social network. Section 3.2 provides an overview of the principles underlying the S3G2 (Scalable Structure-correlated Social Graph Generator) [2] that models huge correlated directed labeled graphs, on which the social network generator is based. We discuss the data generation process in Section 3.3; more specifically the *parameters*, *process*, *dictionaries* and *property-value* generation. Last Section 3.4 presents the statistics for a dataset describing the activities of 100k users during 1 year. We conclude in Chapter 4.

2 SEMANTIC PUBLISHING

The LDBC Semantic Publishing Benchmark (SPB) simulates the management and consumption of RDF metadata that describes media assets, or creative works (CW). The scenario is based around a media organization that maintains RDF descriptions of its catalogue of creative works. The RDF descriptions use an ontology that defines numerous properties for content, for example: date of creation, short/long descriptions, etc. Furthermore, a tagging ontology is used to connect individual creative work descriptions to instances from reference datasets, e.g. sports, geographical, or political information.

2.1 Semantic Publishing: Ontologies

In this section we present the ontologies provided by BBC for developing the *semantic publishing benchmark*. Ontologies are depicted as *node and edge labeled directed* graphs where *classes* are depicted by an *oval*, *class instances* by a *rhombus* and *properties* are depicted as *edges* between classes and instances, where the name of the property is the label of the edge. User defined and RDF properties (rdf:type, rdfs:subClassOf, rdfs:subPropertyOf) are depicted in the same manner. Cardinality constraints for properties are also recorded on the edges at the target class.

CreativeWork 0.9: this ontology defines the *classes* and *properties* for *creative works*. Figure 2.1 shows the classes and properties for the creative works ontology. A creative work (also called a *journalistic asset*) is something created by the publisher's editorial team. It is not a representation of the item itself (which could be text, a photo, a video, an audio recording, etc), rather the *metadata* that describes it and its location (in an appropriate content management system). A creative work has a *title*, *shortTitle*, exactly one description, modification and creation date (properties cwork:description, cwork:dateModified and cwork:dateCreated respectively). Property cwork:liveCoverage indicates that the creative work is the live coverage of an event. It has zero or more audiences (property cwork:audience), instances of class cwork:Audience, a single format (property cwork:primaryFormat), instance of class cwork:Format. Creative works can be tagged (property tag) by *anything* (instance of class core:Thing), and is associated with exactly one category (property cwork:category) that can be any URI. Properties cwork:about and cwork:mentions are subproperties of property cwork:tag. Class cwork:Audience describes the kinds of audience for the story presented by a creative work; instances of this class are cwork:NationalAudience, cwork:InternationalAudience. Class cwork:Format collects all different kinds of formats of a creative work. These are cwork:PictureGalleryFormat, cwork:AudioFormat, cwork:InteractiveFormat, cwork:VideoFormat, cwork:TextualFormat.

A creative work has exactly one thumbnail (property cwork:thumbnail). Thumbnails have at most one type (property cwork:thumbnailType), instance of class cwork:ThumbnailType: namely cwork:StandardThumbnail, cwork:CloseUpThumbnail, cwork:FixedSize66Thumbnail, cwork:FixedSize228Thumbnail and cwork:FixedSize466Thumbnail and a text description (property cwork:altText).

There are different types of creative works: *news article* (class cwork:NewsArticle), a *programme* (class cwork:Programme) and a *blog post* (class cwork:BlogPost); these types are represented using the rdfs:subClassOf RDFS property and are subclasses of cwork:CreativeWork class.

Company 1.4: This ontology describes the relationship between the *web documents* produced by a content management system (class bbc:WebDocument), *BBC products* (class bbc:Product) and *creative works* (class cwork:CreativeWork). A BBC product can be a *blog* (bbc:Blogs), *education* (bbc:Education), *news* (bbc:News), *music* (bbc:Music) or *sport* (bbc:Sport), all instances of bbc:Product. A web document (instance of class bbc:WebDocument) has an associated *product* (property bbc:product). These documents have exactly one *primary topic* (property core:primaryTopic) that can be anything (instance of core:Thing). Such documents are presented in at most one platform such as bbc:Mobile, bbc:HighWeb, instances of class

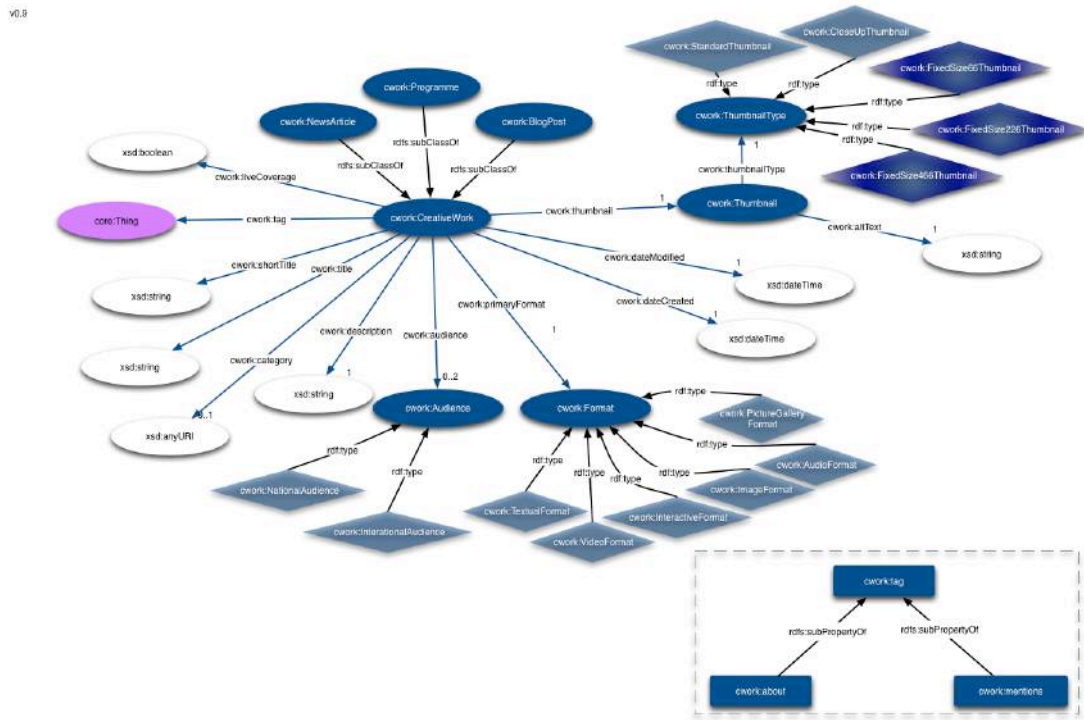


Figure 2.1: Creative Work 0.9

bbc:Platform. A creative work can be the primary content (bbc:primaryContentOf)¹ of at least one and at most two web documents. Finally, a BBC web document has an associated product, the former being the *primary content* of a *creative work*. Figure 2.2 presents the classes and properties of the company ontology.

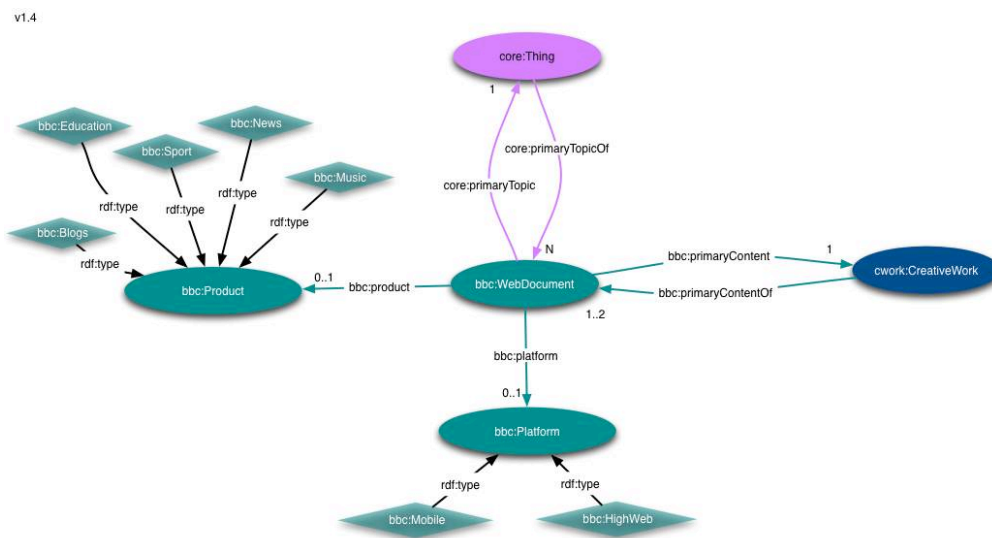


Figure 2.2: Company Ontology 1.4

¹bbc:primaryContent is the *inverse* of bbc:primaryContentOf: a web document can have a single creative work as its primary content.

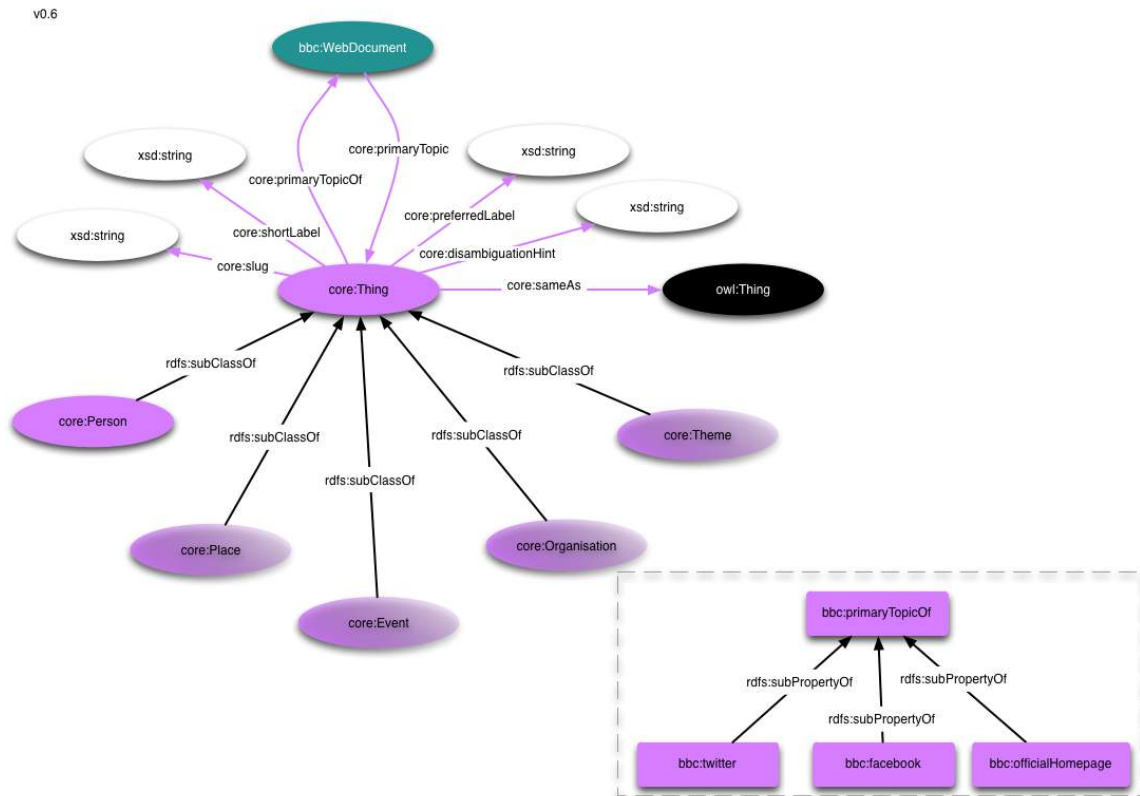


Figure 2.3: Core Concepts Ontology 0.6

Core Concepts 0.6 defines the main classes used in BBC datasets such as `core:Person`, `core:Place`, `core:Event`, `core:Organization` and `core:Theme`. These are all subclasses of the class `core:Thing`. `core:Thing` is defined as equivalent of class (using the `owl:sameAs` property) `owl:Thing`, that is the “class of all individuals in the OWL world”². An instance of class `core:Thing` has *short*, *preferred* labels (properties `core:shortLabel`, `core:preferredLabel`), *disambiguation hint* (property `core:disambiguationHint`). Finally, each instance of class `core:Thing` has an associated URI slug (property `core:slug`) that is the fragment of a URI that uniquely identifies a resource within a domain. For instance, in the case of Wikipedia the URI slug for the entry *Stoat*: <http://en.wikipedia.org/wiki/Stoat> is “Stoat” [3]. `core:primaryTopicOf` property is the *inverse* of `core:primaryTopic`. An instance of `core:Thing` can be the primary topic of more than one web documents. Properties `bbc:twitter`, `bbc:facebook` and `bbc:officialHomepage` are *subproperties* (modeled using the RDFS built-in `rdfs:subPropertyOf` relationship) of `core:primaryTopicOf` that are used to indicate the different kinds of web documents that have a “thing” as primary content. The main classes and properties of the core concepts ontology are shown in Figure 2.3.

CMS 1.2.: the ontology is used for interpreting locators into various specialized content management systems. A creative work and a BBC “thing” can have multiple such locators, that can be *sport-stats* (class `cms:Sports – Stats`), *music bootstrap* (class `cms:MusicBootstrap`), *iscript* (class `cms:iScript`) and *content api* (class `cms:ContentApi`). The CMS classes are all subclasses of `cms:Locator`. The CMS ontology used in BBC is shown in Figure 2.4.

Person 0.2 describes information related to *persons*, instances of class `person:Person`, considered to be a *subclass* of class `bbc:Thing`. A person can have a *role*, a *first* and a *last name* (properties `person:role`, `person:firstName`, `person:lastName`). The person ontology is shown in Figure 2.5.

²<http://www.w3.org/TR/owl-guide/>

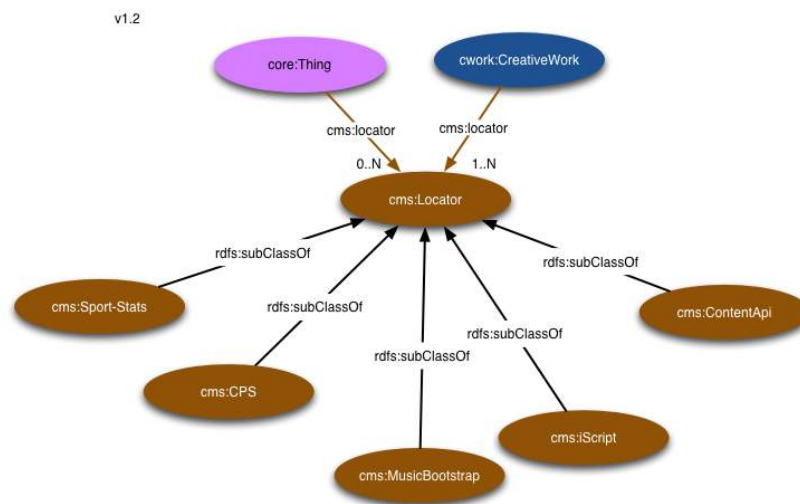


Figure 2.4: CMS Ontology 1.2

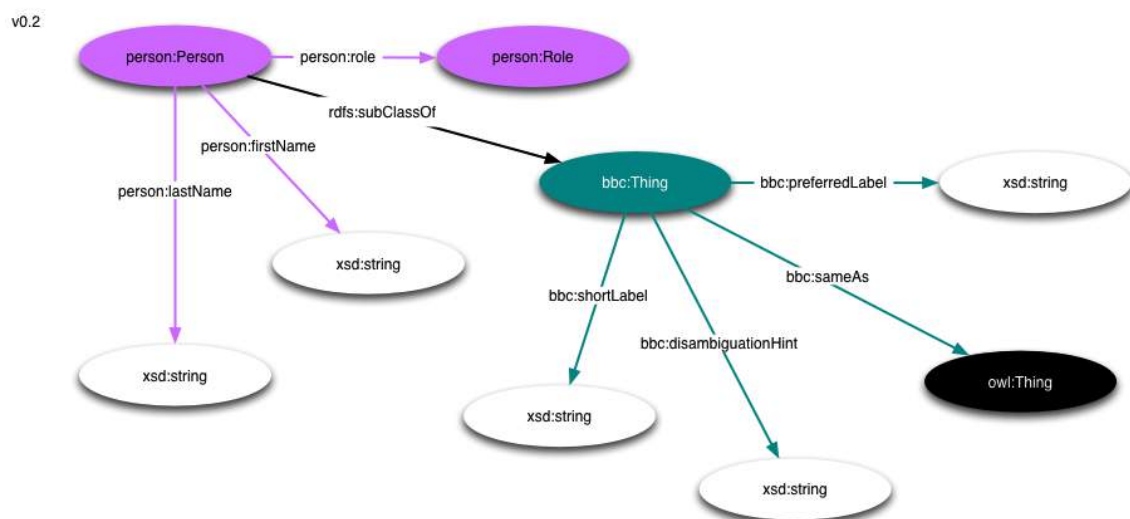


Figure 2.5: Person Ontology 0.2

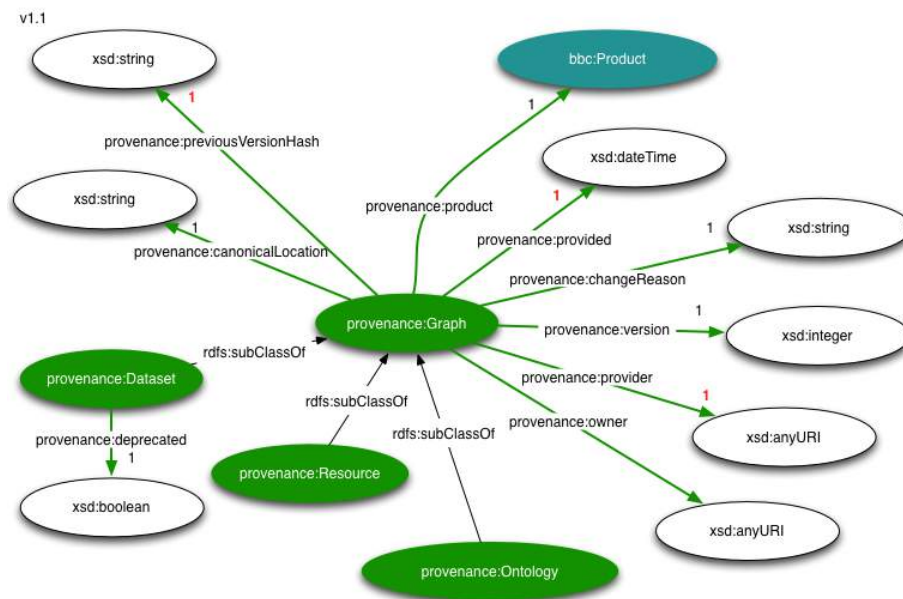


Figure 2.6: Provenance Ontology 1.1

Provenance 1.1 specifies the main concepts and properties used to describe versioning and change log information for the BBC datasets. The main class of the ontology is `provenance:Graph` that carries information about different versions of a dataset. The information that carries a provenance graph are the *owner* and *provider* of the dataset (properties `provenance:owner`, `provenance:provider`) that can be any web resource. The provision date, the reason a dataset changed and its version, a canonical location and a previous hash version (properties `provenance:provided`, `provenance:changeReason`, `provenance:version`, `provenance:canonicalLocation`, `provenance:previousVersionHash`). The ontology is shown in Figure 2.6.

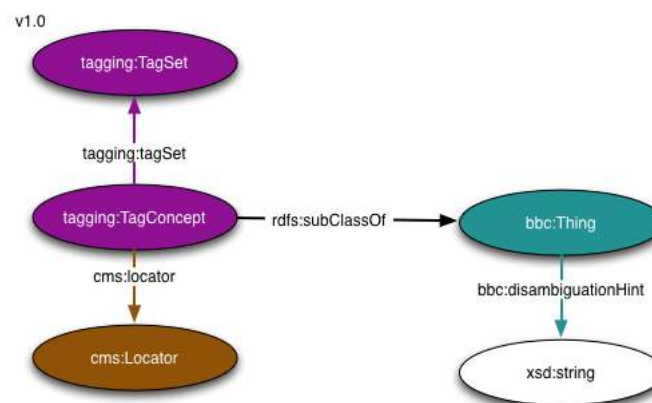


Figure 2.7: Tagging Ontology 1.0

Tagging 1.0: this ontology is used for connecting creative works with concepts from domain ontologies. The main concept is the `tagging:TagConcept` which is a *subclass* of `bbc:Thing`. A tag concept is associated with a *set of tags* (instances of class `tagging:TagSet`) and a locator of a content management system (instance of class `cms:Locator`). The ontology is shown in Figure 2.7.

In addition to the aforementioned general ontologies, concepts from *domain ontologies* are used as tagging

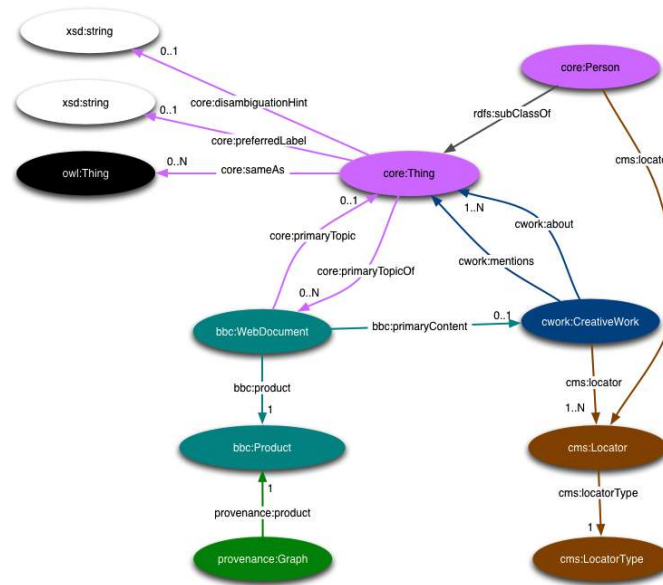


Figure 2.8: Overview Ontology 0.2

concepts: the *sports* ontology contains concepts for describing sports, competitions and sporting events, the *curriculum* ontology describes entities in academia and finally the *news* ontology describes the basic concepts that a *creative work* can be tagged with.

Figure 2.8 presents an overview of the ontologies that comprise the BBC schema. The main classes of each of the ontologies are shown, in a color-coded fashion to indicate the ontology where they come from.

2.2 Semantic Publishing: Workloads

The Semantic Publishing Benchmark models two separate workflows:

Editorial that simulates *creating*, *updating* and *deleting* creative work metadata descriptions for different *stories* (sporting events, elections, music festivals etc.). Media companies use both manual and semi-automated processes for efficiently and correctly managing asset descriptions, as well as annotating them with relevant instances from reference ontologies. For example, a news story about an incident during a football match (i) might be created on a *specific date* (property *cwork:dateCreated*), (ii) have a *title* “George Best hurt during Pitch Invasion at Old Trafford” (property *cwork:title*) (iii) have a *primary format* (property *cwork:primaryFormat*) such as “text” (iv) has an *audience* (property *cwork:audience*) and (vi) is *tagged* to indicate that the story is *about* (property *cwork:about*) a player in the Sports ontology (identified by a URI), and *mentions* (property *cwork:mentions*) a particular football team (identified by a URI). The creative work defined for this news story is shown below:

```
example:story
  rdf:type cwork:NewsItem ;
  cwork:title "George Best hurt during Pitch Invasion at Old Trafford" ;
  cwork:shortTitle "best hurt pitch trafford" ;
  cwork:about person:3107 ;
  cwork:mentions dbpedia:Bristol_City_F.C. ;
  cwork:audience cwork:NationalAudience ;
  cwork:primaryFormat cwork:TextualFormat;
  cwork:dateCreated "2012-12-26T16:36:09.317+02:00"^^
    <http://www.w3.org/2001/XMLSchema#dateTime> ;
```

The editorial workload is specified by means of *insert*, *delete* and *update templates* that the journalists use to create creative works.

Insert Template: Inserts a Creative Work in its own graph

```
INSERT DATA {
  GRAPH {{{cwGraphUri}}} {
    {{{cwUri}}} a {{{cwType}}} ;
    cwork:title {{{cwTitle}}} ;
    cwork:shortTitle {{{cwShortTitle}}} ;
    cwork:category {{{cwCategory}}} ;
    cwork:description {{{cwDescription}}} ;
    {{#{cwAboutsList}}
    cwork:about {{{cwAboutUri}}} ;
    {{/cwAboutsList}}
    {{#{cwMentionsList}}
    cwork:mentions {{{cwMentionsUri}}} ;
    {{/cwMentionsList}}
    cwork:audience {{{cwAudienceType}}} ;
    cwork:liveCoverage {{{cwLiveCoverage}}} ;
    {{#{cwPrimaryFormatList}}
    cwork:primaryFormat {{{cwPrimaryFormat}}} ;
    {{/cwPrimaryFormatList}}
    cwork:dateModified {{{cwDateModified}}} ;
    cwork:thumbnail {{{cwThumbnailUri}}} .
    {{#{cwPrimaryContentList}}
    {{{cwUri}}} bbc:primaryContentOf {{{cwPrimaryContentUri}}} .
    {{{cwPrimaryContentUri}}} bbc:webDocumentType {{{cwWebDocumentType}}} .
    {{/cwPrimaryContentList}}
  }
}
```

Update Template: Updates a Creative Work by first dropping its entire graph and creating a new one.

```
DROP GRAPH {{{cwGraphUri}}} ;
INSERT DATA {
  GRAPH {{{cwGraphUri}}} {
    {{{cwUri}}} a {{{cwType}}} ;
    cwork:title {{{cwTitle}}} ;
    cwork:shortTitle {{{cwShortTitle}}} ;
    cwork:category {{{cwCategory}}} ;
    cwork:description {{{cwDescription}}} ;
    {{#{cwAboutsList}}
    cwork:about {{{cwAboutUri}}} ;
    {{/cwAboutsList}}
    {{#{cwMentionsList}}
    cwork:mentions {{{cwMentionsUri}}} ;
    {{/cwMentionsList}}
    cwork:audience {{{cwAudienceType}}} ;
    cwork:liveCoverage {{{cwLiveCoverage}}} ;
    {{#{cwPrimaryFormatList}}
    cwork:primaryFormat {{{cwPrimaryFormat}}} ;
    {{/cwPrimaryFormatList}}
    cwork:dateModified {{{cwDateModified}}} ;
    cwork:thumbnail {{{cwThumbnailUri}}} .
    {{#{cwPrimaryContentList}}
    {{{cwUri}}} bbc:primaryContentOf {{{cwPrimaryContentUri}}} .
    {{{cwPrimaryContentUri}}} bbc:webDocumentType {{{cwWebDocumentType}}} .
    {{/cwPrimaryContentList}}
  }
}
```

Amount of Tags	(A)	(B)	(C)
1	22.33%	10.06%	94.77%
2	32.67%	23.13%	3.82%
3	24.60%	30.88%	0.93%
4	11.63%	22.78%	0.31%
5	3.27%	10.35%	0.12%
6	1.52%	2.80%	0.05%
7	1.00%	0%	0%
8	0.74%	0%	0%
9	0.69%	0%	0%
10	0.47%	0%	0%

Table 2.1: Distribution of tags in creative works

Aggregation queries simulate the dynamic aggregation of content for consumption by the distribution pipelines, e.g. a web-site. The publishing activity is described as *dynamic* because the content is not manually selected and arranged on, say, a web page. Instead, templates for pages are defined and the content selected at the moment when a consumer accesses the page. In this workflow, SPARQL queries are used to find relevant content, e.g. the most recent stories and photos about a specific football player. This allows for a rich, up to date pages to be rendered that contain relevant content from a range of media types, as well as relevant links to further content and pages.

The aggregation and editorial templates use the Mustache Template System³ a cross-platform templating system, to replace values in the corresponding template fields.

2.3 Semantic Publishing: Data Generator

The semantic publishing generator takes into account the *ontologies* presented in Section 2.1 and the *reference datasets*. The reference datasets are snapshots of real datasets provided by BBC about (1) international and national (in particular Scottish and English) football competitions and teams, (2) Formula One competitions and teams and (3) place names from the GeoNames dataset [4].

Data generation was dictated by the BBC query workload discussed briefly in Section 2.2. Recall, that the aggregation queries retrieve creative works using the tags used in the `cwork:about` and `cwork:mentions` properties. Hence, the primary objective of the data generator was to reproduce the distribution of the `cwork:about` and `cwork:mentions` tags in the generated data, based on the statistical distribution in the available reference datasets. The distribution statistics used are shown in Table 2.1 and their charts in Figure 2.9.

These tags are obtained from the *popular instances* (instances of class `tagging:TagConcept`) of the reference datasets. The information extracted from these instances to be used for tagging the creative work instances are (i) instances of classes in BBC ontologies such as `sport:CompetitiveSportingOrganisation`, `sport:RecurringCompetition`, `news:Person` and (ii) values of properties of those instances, for instance `domain:canonicalName`, `bbc:preferredLabel`, `bbc:sameAs` and `rdfs:seeAlso`. The query that is used to obtain these tags is shown below:

```
SELECT DISTINCT ?uri ?label ?category ?location {
{
?uri a sport:CompetitiveSportingOrganisation .
?uri domain:canonicalName ?label .
BIND("SportsTeams" AS ?category) .
```

³<http://mustache.github.io/>

```

}
UNION
{
?uri a sport:RecurringCompetition .
?uri domain:canonicalName ?label .
BIND("SportsCompetitions" AS ?category) .
}
UNION
{
?uri a news:Person .
?uri bbc:preferredLabel ?label .
BIND("PoliticsPersons" AS ?category) .
}
UNION
{
?tempUri a news:Person ;
bbc:preferredLabel ?label ;
bbc:sameAs ?uri .
BIND("PoliticsPersonsAdditional" AS ?category) .
}
UNION
{
?tempUri a news:Person ;
bbc:preferredLabel ?label ;
rdfs:seeAlso ?uri .
BIND("PoliticsPersonsReference" AS ?category) .
}
}

```

Instances determined with this process are grouped into two lists - a list of "popular" instances with randomly chosen 5% of all instances, and a list of "regular" (or "not popular") instances - the rest 95% of all instances.

In subsequent versions of the benchmark, other methods could be used for this purpose (e.g., the number of statements of a reference instance) but we expect that those variations will not be significant. Data generation is done by indirectly using inserts from the Editorial workload described in Section 2.2.

The result of the data generation process is a set of RDF triples (which can be saved in various serialization formats) that describe creative works (instances of class `cwork:CreativeWork`) where the values of properties `cwork:about` and `cwork:mentions` are the tags of instances obtained from the previous process. The value for each property is then selected as follows: 30% of the "about/mentions" taggings will come from randomly selecting an instance stored in the "popular" instances list (the "popular" instances list containing 5% of all retrieved instances), and the rest 70% will come from randomly selecting an instance from the "regular" instances list (the "regular" instances list containing 95% of all retrieved instances). That way a bias towards more-popular instances will be created in 30% of all generated creative works.

Figure 2.9 depicts the charts for Table 2.1 above. Column (A) in Table 2.1 (chart 2.9(a)) presents the percentage of creative works that have 1, 2, .. 10 tags (`cwork:about` and `cwork:mentions` tags) over the set of creative works. Columns (B) and (C) present the same distribution for `cwork:about` and `cwork:mentions` tags.

Additionally the data generator can be fine-tuned for experimenting with different dataset allocations, by changing various configuration properties (saved in file `definition.properties`), e.g. change *about* or *mentions* tags properties allocations or change the *popularity ratio* and *popularity bias* ratio for tagged entities. The `definition.properties` file contains a set of allocation parameters that can be altered in order to perform additional fine-tuning to the data generator. Some of these parameters apply to the agents of the editorial workload discussed in Section 2.2. We present here the parameters used for the data generator only:

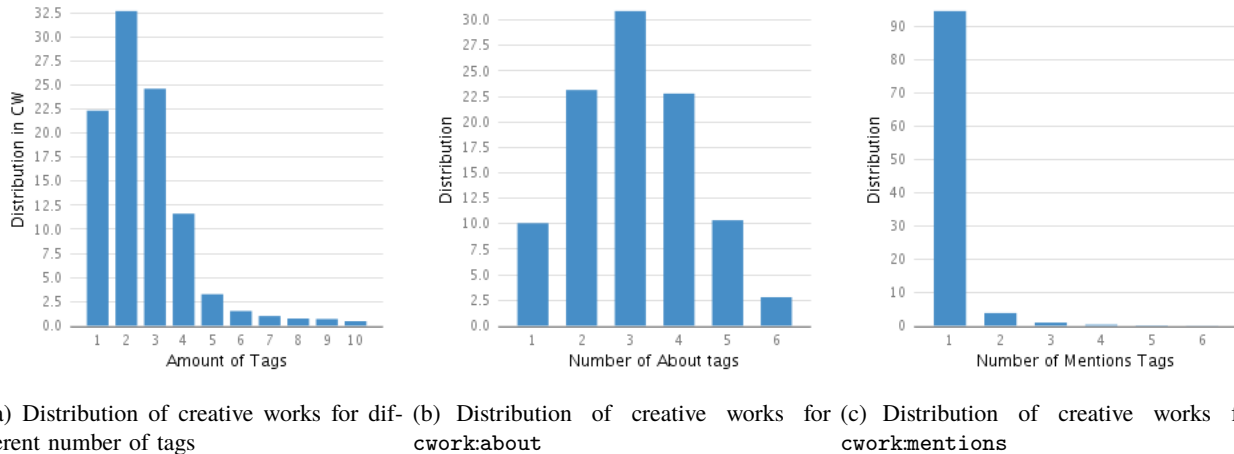


Figure 2.9: Distribution of tags for Creative Works

1. *aboutAllocations* and *mentionsAllocations* define the number of `cwork:about` and `cwork:mentions` tags respectively of creative works (instances of class `cwork:CreativeWork`). The *creativeWorkTypeAllocation* specify the allocation of different types of creative works `news:BlogPost`, `news:NewsItem` and `news:Programme`).
2. *entityPopularity* specify the amount of *entities* to be considered as popular among all entities found in the reference datasets. On the other hand, *usePopularEntities* define the *amount of tags* that will use popular entities during data-generation (and aggregation).

In addition to the above parameters, the following are also considered when producing information regarding creative works:

1. randomly generated and sized sentences used for the `cwork:title`, `cwork:shortTitle` and `cwork:description` properties.
2. randomly generated date-time information (`cwork:dateModified` property) which is within a range of one year from the current date.
3. 45% of creative works have a type: `cwork:BlogPost` (35%), `cwork:NewsItem` (20%), `cwork:Programme` (45%).
4. the type of `cwork:audience` is chosen on the basis of the type of creative work. More specifically, for instances of `cwork:BlogPost` the audience is `cwork:InternationalAudience` and for `cwork:NewsItem` the audience is `cwork:NationalAudience`.
5. the value of the `cwork:liveCoverage` is a boolean based on the type of the generated creative work.
6. the value of `cwork:primaryFormat` is one of the values `cwork:TextualFormat`, `cwork:InteractiveFormat`, `cwork:VideoFormat` and `cwork:AudioFormat`.
7. the value of `cwork:thumbnail` is a randomly generated URI; `cwork:altText` value is a random text string and is produced in the case in which the `cwork:thumbnail` property cannot be resolved.
8. property `cwork:primaryContentOf` refers to a randomly generated URI of a web document; the assumption is that such a document exists and is identified by the respective URI.

2.4 Statistics

The statistics for a dataset of 50M triples produced by the Semantic Publishing Data Generator are shown in Tables 2.2 - 2.11. We also provide the SPARQL queries used to generate the statistics.

CREATIVE WORKS	2533634
QUERY	<pre>select (count(*) as ?cwCount) { ?cw a cwork:CreativeWork . }</pre>

Table 2.2: *Creative Works*

BLOGPOST	1140041	45.00%
NEWSITEM	88707	35.01%
PROGRAMME	506512	19.99%
QUERY	<pre>select ?cwType (count(*) as ?cwTypeCount) { ?cw a cwork:CreativeWork . ?cw a ?cwType . } group by ?cwType order by ?cwTypeCount</pre>	

Table 2.3: Statistics for *Creative Work Type*

SPORTSTEAMS	113142	4.47%
SPORTSCOMPETITIONS	22917	0.90%
POLITICSPERSONS	353343	13.95%
POLITICSPERSONSADDITIONAL	1026925	40.53%
POLITICSPERSONSREFERENCE	1017297	40.15%
QUERY	<pre>select ?categoryType (count(*) as ?categoryTypeCount) { ?cw a cwork:CreativeWork . ?cw cwork:category ?categoryType . } group by ?categoryType order by ?categoryTypeCount</pre>	

Table 2.4: Statistics for *Creative Work Category*

INTERNATIONALAUDIENCE	1646553	64.99%
NATIONALAUDIENCE	887071	35.01%
QUERY	<pre>select ?cwAudienceType (count(*) as ?cwAudienceTypeCount) { ?cw a cwork:CreativeWork . ?cw cwork:audience ?cwAudienceType . } group by ?cwAudienceType order by ?cwAudienceTypeCount</pre>	

Table 2.5: Statistics for *Audience Type*

TEXTUALFORMAT	2027112	80.01%
INTERACTIVEFORMAT	1457400	57.52%
AUDIOFORMAT	253336	10.00%
VIDEOFORMAT	253176	9.99%
QUERY	<pre> select ?cwPrimaryFormatType (count(*) as ?cwPrimaryFormatTypeCount) { ?cw a cwork:CreativeWork . ?cw cwork:primaryFormat ?cwPrimaryFormatType . } group by ?cwPrimaryFormatType order by ?cwPrimaryFormatTypeCount </pre>	

Table 2.6: Statistics for *Primary Format*

TRUE	506512	19.99%
FALSE	2027112	80.01%
QUERY	<pre> select ?cwLiveCoverageType (count(*) as ?cwLiveCoverageCount) { ?cw a cwork:CreativeWork . ?cw cwork:liveCoverage ?cwLiveCoverageType . } group by ?cwLiveCoverageType order by ?cwAudienceTypeCount </pre>	

Table 2.7: Statistics for *Live Coverage* (count)

1	254941	10.06%
2	586287	23.14%
3	782666	30.89%
4	576462	22.75%
5	262700	10.37%
6	70568	2.79%
QUERY	<pre> select ?cwCount (count(*) as ?cwAboutsCountSize) { select (count(*) as ?cwCount) { ?cw a cwork:CreativeWork . ?cw cwork:about ?cwAbout . } group by ?cw } group by ?cwCount order by ?cwCount </pre>	

Table 2.8: Statistics for *About* (count)

1	2401076	94.77%
2	96798	3.82%
3	23711	0.94%
4	7833	0.31%
5	2976	0.12%
6	1230	0.05%
QUERY	<pre> select ?cwCount (count(*) as ?cwMentionsCountSize) { select (count(*) as ?cwCount) { ?cw a cwork:CreativeWork . ?cw cwork:mentions ?cwAbout . } group by ?cw } group by ?cwCount order by ?cwCount </pre>	

Table 2.9: Statistics for *Mentions* (count)

1	844052	33.31%
2	845230	33.36%
3	844342	33.33%
QUERY	<pre> select ?primaryContentNumber (count(*) as ?primaryContentSize) { select (count(*) as ?primaryContentNumber) { ?cw a cwork:CreativeWork . ?cw bbc:primaryContentOf ?primaryContentOf . } group by ?cw } group by ?primaryContentNumber order by ?primaryContentNumber </pre>	

Table 2.10: Statistics for *Primary Content* (count)

HIGHWEB	2533438	99.99%
MOBILE	2534100	100.02%
QUERY	<pre> select ?webDocumentType ?webDocumentNumber { select ?webDocumentType (count(*) as ?webDocumentNumber) { ?cw a cwork:CreativeWork . ?cw bbc:primaryContentOf ?primaryContentOf . ?primaryContentOf bbc:webDocumentType ?webDocumentType . } group by ?webDocumentType } group by ?webDocumentNumber ?webDocumentType </pre>	

Table 2.11: Statistics for *Web Document Types*

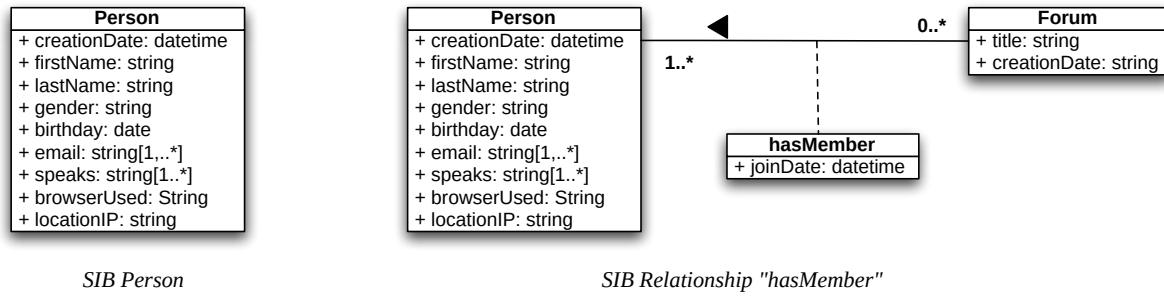


Figure 3.2: SNDG Entity and Relationship

record the cardinality of a class in a relationship - a class can participate in zero or one (0..1), one only (1), zero or more (0..*), one or more (1..*) times in a relationship. A relationship can also have attributes (called *attributed relationship*); such attributes provide valuable information about the relationship. Such a relationship is represented by means of an association class. Figure 3.2 shows entity *Person* - the basic entity of the SNDG schema, and relationship *hasMember* defined in entity *Forum* discussed in detail below. According to the UML diagram, an instance of class *Forum* can have *at least one* instance of class *Person* as member (cardinality (1, *)) whereas a person can be a *member of* zero or more forums (cardinality (0, *)).

The notion of inheritance can be used to model a hierarchy of entity classes. Inheritance is represented with a solid line drawn from the child class (subclass), with an unfilled arrowhead, pointing to its parent class (superclass). Figure 3.3 presents class *Organization* and its subclasses *University* and *Company*. Finally, in some cases, inheritance can be used to simplify the representation of two or more entity classes sharing some attributes and relationships. An interface is a *virtual* entity class, which represents inheritance but the database does not contain data instances of such class. Figure 3.3 shows an example of the *Message* interface defined in the SNDG schema.

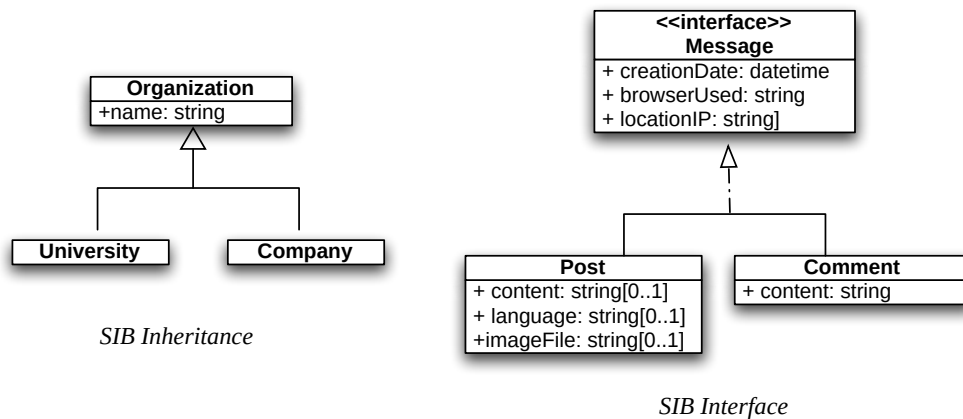


Figure 3.3: SNDG Inheritance and Interface

An instance of a social graph is comprised of *objects* in the graph that are instances of the entities depicted by boxes in the schema shown in Figure 3.4.

The main concept in the SNDG schema is *Person*. Instances of this concept are *avatars* of real world persons and these are created when a person joins the network. The avatars record information regarding the *real world person* this avatar represents, as well as network information. Attributes *firstName*, *lastName*, *gender*, *birthday*, *email* and *speaks* fall into the first category whereas *browserUsed*, *locationIP*, *creationDate* fall into the second. The first connection of the person to the Social Network registers the IP address and

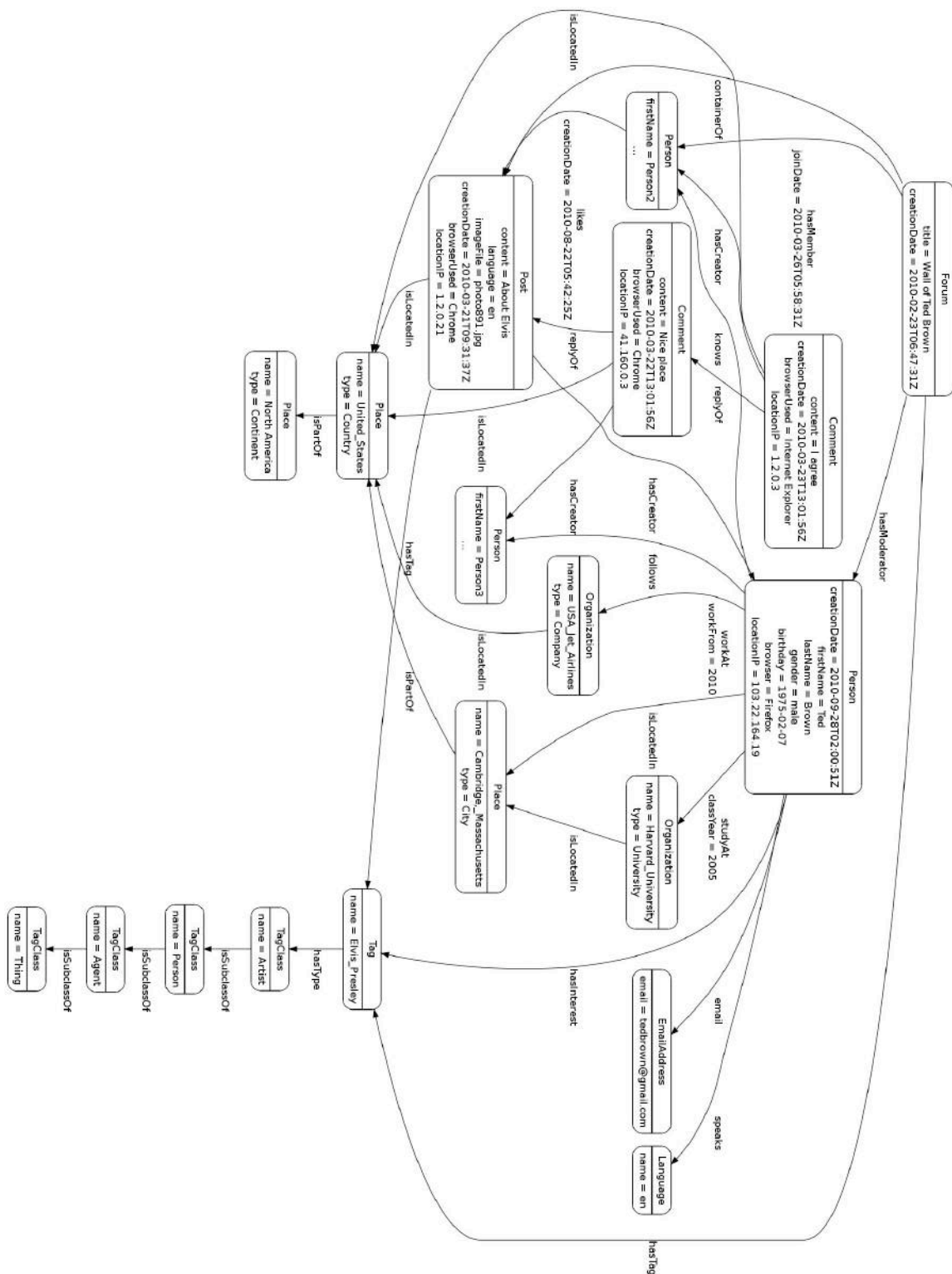


Figure 3.4: An example of SNDG

the browser used. A person has one or more *email* addresses, *speak* one or more languages that are official or commonly spoken in her country, or even foreign languages. People in the social graph are connected to others by establishing *friendships*. The facts that a person may know and follow zero or more persons, are recorded by the relationships *knows* (symmetric relationship) and *follows*. Each person is *locatedIn* a city. There is a hierarchy of *places*: concepts *City*, *Country* and *Continent* are defined as subconcepts of *Place*. The SNDG schema records the regional correlation between the entities using the transitive *partOf* relationship.

A person has studied (relation *studyAt*) and worked (relation *workAt*) at zero or more universities and companies respectively. The former records the graduation year, and the latter the year the person started working for the company. Universities and companies are instances of entities *University* and *Company* respectively, subentities of *Organization* and located in (attribute *isLocatedIn*) a city and a country respectively.

A person has zero or more interests (attribute *hasInterest*), each interest (instance of entity *Tag*) can have different types (entity *TagClass*) that form a hierarchy of types. A person is the moderator of a forum (entity *Forum*). A forum can have at most one moderator (this is recorded by attribute *hasModerator*), and can have at least one person as member (indicated by relationship *hasMember*). It also has a title (attribute *title*) and date of creation (attribute *creationDate*). The SNDG schema records the date (attribute *joinDate*) that the person has joined the forum. A forum can be tagged with zero or more tags that record the topics of interest of the forum. A forum might have at least one published post (entity *Post*, attribute *containerOf*). A post has a *content*, is written in some *language* and/or is associated with an image (attribute *imageFile*). Similar to forums, posts are tagged (attribute *tag*) with zero or more tags. Persons *like* zero or more posts - the date the person indicated that she likes the post is recorded using the *creationDate* attribute. People can comment posts where a comment is just a simple text (attribute *content*). Posts and comments are messages and carry information such as the creation date (attribute *creationDate*), the browser used (attribute *browserUsed*) and the IP address (attribute *locationIP*). Comments can reply to zero or more posts and comments (attribute *replyOf*).

3.2 Social Network Data Generator

An important component of the Social Network benchmark is the data generator used to produce *synthetic data* that mimic the characteristics of the real data. The Social Network Data Generator is based on the S3G2 data generator [2]. S3G2 (Scalable Structure-correlated Social Graph Generator) targets at modeling huge correlated directed labeled graphs. S3G2 and hence SNDG, use a *graph model* that is essentially a *correlated property graph*. Definition 1 of Pham et. al [2] describes this graph structure.

Definition 1 *S3G2 produces a graph $G(V, E, P, C)$ where V is a set of nodes, E is a set of edges, P is a set of properties and C is a set of classes.*

$$\begin{aligned} V &= L \cup \bigcup_{c \in C} O^c \\ P &= \{P^{L(x)} | x \in C\} \cup \{P^{E(x,y)} | x, y \in C\} \\ E &= \{(n_1, n_2, p) | n_1 \in O^x \wedge ((n_2 \in L \wedge p \in P^{L(x)}) \vee (n_2 \in O^y \wedge p \in P^{E(x,y)}))\} \end{aligned}$$

where O^c is an object of class c in C ; L is the set of literals; $P^{L(x)}$ is set of literal properties of class x in C ; $P^{E(x,y)}$ is the set of properties representing relationship edges that go from instances of class x to class y .

The building blocks of the data generator are *property dictionaries*, *simple subgraph generation*, and *edge generation along correlation dimensions*.

3.2.1 Property Dictionary

Property values for each literal property are generated following a *property dictionary model* that is defined by i) a *dictionary* D , a *ranking function* R and a *probability density function* F .

Dictionary D is a *fixed set of values* (e.g., terms from DBPedia³, GeoNames⁴). The ranking function R is a bijection that assigns to each value in a dictionary a unique *rank* between 1 and $|D|$. The probability density function specifies how the data generator chooses values from dictionary D using the rank for each term in the dictionary. The idea to have a separate ranking and probability function is motivated by the need of generating correlated values: in particular, the ranking function is typically parameterized by some parameters: different parameter values result in different rankings. For example, in the case of a dictionary of property *firstName*, the popularity of first names, might depend on the *gender*, *country* and *birthDate* properties. Thus, the fact that the popularity of first names in different countries and times is different, is reflected by the different ranks produced by function $R()$ over the full dictionary of names.

The S3G2 data generator contains a dictionary for each literal property, as well as ranking functions for all literal properties (independently of the class they are defined in). It is important that when designing correlation parameters for a ranking function, the amount of parameter combinations (see the example above) stays limited, in order to keep the representation of such functions compact.

3.2.2 Graph Generation

The idea behind a simple graph generation algorithm is to create new edges and nodes in one pass, starting from an existing node n and start creating new nodes to which n will be connected. The degree distribution function is used to guide the above process by determining the number of descendants that a node will have. It has been observed that in a number of social networks, the amount of edges that start from a node, follows a *power law* distribution. In S3G2 it is possible to have such a degree distribution function from which the degree of each node is generated, correlated with properties of the node. For instance, people with many friends in a social network will typically post more pictures than people with few friends (hence, the amount of friend nodes in our use case influences the amount of posted comment and picture nodes in the graph). The disadvantage of this process is that it might lead to isolated graphs that are simply dangling from node n the whole process was initiated from.

The S3G2 data generator generates correlated and highly connected graph data by creating edges *after* generating many nodes. This approach, when compared to a naive graph generation algorithm, is computationally harder than generating edges towards new nodes. The reason is that if node properties influence their connectivity, a naive implementation would have to compare the properties of all existing nodes with all nodes, something that would lead to quadratic computational cost and a random access pattern, so the generation algorithm would only be fast as long as the data fits in RAM (to avoid a random I/O access pattern).

The use of data correlation provides a solution to this problem: the probability that two nodes are connected is typically skewed with respect to some similarity between the nodes. That is, for a node n and for a small set of nodes that are somehow similar to it, there is a high connectivity probability, whereas for most other nodes, this probability is quite low. This knowledge can be exploited by a data generator by identifying correlation dimensions.

For a certain edge label $e \in P^{E(x,y)}$ between node classes O^x and O^y , a correlation dimension $CD_e(M^x, M^y, F)$ consists of two *similarity metric* functions $M^x : n \rightarrow [0, \infty]$, $M^y : n \rightarrow [0, \infty]$, and a probability distribution $F : [1, W.t] \rightarrow [0, 1]$, where $W.t$ is a window size, of W tiles with each t nodes. For instance, in the case of friends in a social network, both start and end of the edges are of the class person ($O^x = O^y$), so a single metric function would typically be used. The description below assumes that $M = M^x = M^y$.

The similarity metric is computed by invoking the similarity function M on all nodes; nodes are subsequently sorted on this score. The consequence of this approach is that the similar nodes are grouped together, and consequently, the larger the distance between two nodes indicates a monotonic increase in their similarity difference. In order to choose the nodes to connect, S3G2 uses a geometric probability distribution for F that provides a probability for picking nodes to connect with that are between 1 and $W.t$ positions apart in

³DBPedia: <http://el.dbpedia.org/>

⁴GeoNames: <http://www.geonames.org/>

this similarity ranking [2]. In order for F to be a geometric distribution, S3G2 should not cut short at $W.t$ positions apart, but consider even further apart nodes. The advantage of this window shortcut is that after sorting the data, it allows S3G2 to generate edges using a fully sequential access pattern that needs little RAM resources since it only buffers $W.t$ nodes.

Similarity functions and probability distribution functions over ranked distance drive *what* kind of nodes will be connected with an edge, not *how many*. This decision on the degree of a node is made before generating the edges, using the *degree function* N (recall that in social networks this would be a power-law function). The edges that will be connected to a node n (i.e., the edges that determine the degree of a node), are selected by randomly picking the required number of edges according to the correlated probability distributions as discussed before. In the case that multiple correlations exist, another probability function is used to divide the intended number of edges between the various correlation dimensions. Thus, S3G2 has a power-law distributed node degree, and a predictable (but not fixed) average split between the reasons for creating edges.

Another parameter that is taken into consideration when generating the graph is *random dimension*. Generating edges between the most similar nodes in all the correlation dimensions is very restrictive: unlikely connections in a social network that the data model would not explain or make plausible, will occur in practice. This random noise is introduced in the generated data by creating a special correlation dimension where a random function can be used as a similarity metric along with a uniform probability function.

3.2.3 Implementation

The S3G2 data generator algorithm is implemented using a MapReduce algorithm that is built on the building blocks discussed earlier: (a) correlated data dictionaries, (b) naive graph generation and (c) edge generation using correlation dimensions. The idea is that a Map function runs on different parts of the input data, in parallel and on many clusters. This function processes the input data, produces for each result a key. Reduce functions then obtain this data and Reducers run in parallel on many cluster machines. The produced key simply determines the Reducer to which the results are sent.

This key is also used to sort the data for which edges will be generated according to the previously discussed similarity metric of the correlation dimension to be used next. Since there are many correlation dimensions, there will be multiple successive MapReduce phases. Map and Reduce functions can perform simple graph generation as previously discussed in addition to generating correlated property values using dictionaries.

The main task of the Reduce function is to (a) sort on the correlation dimension (b) generate edges between nodes using an algorithm that is based on *sliding windows*. The idea behind this approach to correlated edge generation is that when generating edges, only the edges that connect similar (according to the similarity metric) nodes should be considered. By ordering these nodes, the algorithm can keep a small window of nodes (properties and edges) in RAM. Subsequently, only the nodes that are cached in this window are considered for edge generation. Note that different correlations can influence the generation of an edge. To handle this, multiple sorts and sequential sliding window passes are needed, leading to multiple MapReduce jobs.

The sliding window approach is implemented by dividing the sorted nodes conceptually in tiles of t nodes: when a Reducer gets a data item, then it adds it to the tile that is processed (an in memory data structure). When the tile is full and there are W tiles in memory, then the oldest tile is dropped. Before it is dropped, the Reduce function generates edges for all the nodes in the tile, implementing the windowing approach and generating edges along a correlation dimension.

In principle, simple graph generation only requires local information (the current node), and can be performed as a Map task, but also as a post-processing job in the Reduce function. Note that node generation also includes the generation of the (correlated) properties of the new nodes. It comes without saying that data correlations introduce *dependencies*, that impose constraints on the order in which generation tasks have to be performed. So, the correlation rules one introduces, naturally determine the amount of MapReduce jobs needed, as well as the order of actions inside the Map and Reduce functions.

3.3 Data Generation

In this section we will present (a) the *parameters* (b) *dictionaries* (c) *correlated dimensions* and discuss the process for generating a social network.

PARAMETERS

The generator requires the definition of the following input parameters:

- the total number of users: `numtotalUser`: long
- the start date `startYear`: date
- the end date `numYears`: date
- the output of the generator `serializerType`: `ttl|csv`
- the minimum number of tags for a person `minNumTagsPerUser` and `maxNumTagsPerUser` set by default to 1 and 10 respectively.

PROCESS

The process of data generation includes the following steps:

1. Initialize parameters and dictionaries.
2. Create persons including relations with universities and companies (correlated by location).
3. Generate interest/tags of persons (correlated by location).
4. Generate friendship relationships (university-country correlated friendships, interest correlated friendships, and random friendships).
5. Generate forums, posts and comments - *messages* - (correlated with the interests of the person).
6. Serialize the generated data (including static data about Places and TagClasses).

DICTIONARIES

In this section we present the data dictionaries used by the Social Network Data Generator. The description of each dictionary is complemented with an example of its content. We should mention that these dictionaries were constructed by using data available in DBpedia.

- (I) `browsersDic` contains the *browser name* and its *popularity probability*. It is used to assign browsers to the persons based on the popularity probability.

Browser	Popularity Probability
Chrome	0.279
Internet Explorer	0.232
Firefox	0.422

- (II) `companiesByCountry` contains the *country* and the *name* of a *company* for that country. It is used to assign a workplace to persons by taking into consideration their *location* (if this information is available).

Country	Company
Afghanistan	KamAir
Afghanistan	BalkhAirlines
Afghanistan	KhyberAfghanAirlines

- (III) `countryAbbrMapping` contains the *country name* and its *abbreviation*. It is used to link countries to IP addresses.

Abbreviation	Country	Description
ac	United Kingdom	Academic Institutions
ad	Andorra	
ar	United Arab Emirates	

- (IV) `dicCelebritiesByCountry` contains the *countryId*, the *celebrityId* and its *cumulated probability of popularity* within that country. The dictionary is used to assign a country's *celebrity* to persons located in the country.

countryId	celebrityId	Probability
0	0	0.27328605200945627
0	1	0.4884160756501182
0	2	0.649645390070922

- (V) `dicLocation` contains the names of *continent*, *country*, *latitude* and *longitude* values, *population* and cumulated *probability* of population. It is used to create the *continent-country* hierarchy and to distribute the user nationality according to the population data.

Continent	Country	Langitude	Longtitude	Population	Probability
Asia	Afghanistan	35	69	15500000	0.0028010447
Africa	Algeria	37	3	29100867	0.008059937
Africa	Angola	−9	13	5646177	0.0090802721

- (VI) `dicTag` contains information about tags (or topics), including *tagId*, *tagClassId*, *dbpediaName*, and *tagName*. This dictionary is a subset of resources `owl:Thing` in DBpedia. *tagName* is the label of the tag. The *dbpediaName* is the label of the tag occurring at the end of its DBpedia URI, and is used to reconstruct such URI. It is used in the serialization part of the software to assign names to the tags and write the basic tag class.

tagId	tagClassId	tagName	dbpediaName
0	349	Hamid_Karzai	Hamid Karzai
1	211	Rumi	Jalal ad-Din Muhammad Rumi
2	98	Mahmud_of_Ghazni	Yamin al-Dawlah Abul-Qasim Mahmud Ibn Sebuk Tegin

- (VII) `email` contains the *email domain name* and the *probability* for the most popular ones; it records only the name for the remaining ones. It is used to assign email domains to users.

Email Domain Name	Probability
gmail.com	0.45
gmx.com	0.20
yahoo.com	0.18

- (VIII) `givennameByCountryBirthPlace` dictionary contains the *country name*, *persons' first names*, *gender*, *birthdate* period and an unused number; it is used to assign a first name to a user according to *gender* and *age* values.

Country	First Name	Gender	BirthDate Period
Abkhazia	Diana	0	0
Abkhazia	Maya	0	0
Abkhazia	Diana Gurtskaya	1	0
Abkhazia	Diana	0	1

In the Table above, the value of the field "birthdate period" defines two periods: 0 stands for the period between 1980 to 1985 and 1 for the period between 1985 to 1990. This encoding is used during the generation of persons to assign first names.

- (IX) `institutesCityByCountry` dictionary contains the *country name*, the *university name* and the *city* where the university is located. It is used to create the *country-city* hierarchy and to assign to the user a university from the same country.

Country	University	City
Aland_Islands	Aland University of Applied Sciences	Mariehamn
Abkhazia	Abkhazian State University	Sukhumi
Afghanistan	Paktia University	Gardez
Afghanistan	Baghlan University	Puli_Khumri

- (X) `languagesByCountry` contains the *name of the country* and data for the *spoken languages*. According to the ISO 639-1 code, * is used to denote an official language and the speaker percentage (0 if unknown). It is used to assign languages of its country to the user.

Country	Language Data
Aruba	es 12.6 en 7.7 nl * 5.8
Antigua and Barbuda	en * 0
United_Arab_Emirates	ar * 0 fa 0 en 0 hi 0 ur 0

- (XI) `popularPlacesByCountry` dictionary contains popular cities in countries. Specifically, it contains the *country name*, the *city name* from DBpedia (including underscores), the *normalized city name* (without underscores), and the *latitude* and *longitude* of the city. The name with underscores is used to "re-construct" the URI of the city in DBpedia.

Country	Country (no spaces)	Location	Latitude	Longitude
Afghanistan	Ab-Kol	Ab Kol	36.22000122070312	68.5
Afghanistan	Ab_Bazan	Ab Bazan	36.93333435058594	69.94999694824219
Afghanistan	Ab_Daw	Ab Daw	36.25	71.16666412353516
Afghanistan	Ab_Gaj	Ab Gaj	36.98333358764648	72.69999694824219

- (XII) `surnameByCountryBirthPlace` dictionary contains the *last names*, the number of their *appearances* and the *country name*. It is used to assign a last name to persons.

#Appearances	Last Name	Country Name	Last Name
2		Abkhazia	Gurtskaya
1		Abkhazia	Kopitseva
1		Adjara	Vashalomidze
7		Afghanistan	Zaland

- (XIII) **tagClasses** dictionary contains the tag classes that have been obtained from DBpedia. For each tag class we have: *id*, *dbpediaName*, and the *label*. The *dbpediaName* is the name occurring at the end of the DBpedia URI of the class. e.g., <http://dbpedia.org/ontology/BasketballLeague>. It is used to *serialize* the name and label of the tag classes.

tagClassId	tagName	label
0	Thing	thing
1	BasketballLeague	basketball league
2	LunarCrater	lunar crater
3	MilitaryPerson	military person

- (XIV) **tagHierarchy** dictionary contains the hierarchy of *tagClasses* used by the generator. Such hierarchy was obtained from DBpedia, by obtaining all the subclasses starting from the class *owl:Thing* (i.e by following relation *rdfs:subClassOf*). The values for *tagClass* and *parentTagClass* properties point to the **TagClasses** dictionary .

tagClassId	parent tagClassId
19	179
136	338
173	211
230	149

- (XV) **tagText** dictionary contains the *tagId* and *text*. It is used to assign a text to the post and comments related to its tags. The texts were obtained by Wikipedia.

tagId	Text
0	David Cameron, David William Donald Cameron (born 9 October 1966) is the Prime Minister of the United Kingdom, First Lord of the Treasury, . . .
1	Gordon Brown, James Gordon Brown (born 20 February 1951) is a British Labour Party politician who was the Prime Minister of the United Kingdom . . .
3	Tony Blair, Anthony Charles Lynton Blair (born 6 May 1953)[1] is a British Labour Party politician who served as the Prime Minister of the United Kingdom

- (XVI) **tagMatrixId** dictionary contains the identifiers of two tags *t1*, *t2* (i.e., tag identifiers) the cumulative percentage for *t1* and the number of references (i.e., same text) the two tags appear together. It is used to select a list of *correlated tags* of the main interest of the user.

Tag <i>t1</i>	Tag <i>t2</i>	Cumulative % <i>t1</i>	# References
2909	4870	0.0	8.0
2909	4871	$2.392072671167751E - 4$	8.0
2909	4872	$4.784145342335503E - 4$	2.0

- (XVII) **ipzones** directory contains one dictionary per country; each dictionary contains a list of valid IP addresses per country. A sample of such a file for country Andorra is shown below:

85.94.160.0/19
91.187.64.0/19
109.111.96.0/19
194.158.64.0/19

PROPERTY-VALUE GENERATION

Below we discuss the the *property-value* correlations used in the social network data generator.

Entity Person

- The value of a person's identifier (*id*) is a unique number (integer) that is produced sequentially ($\text{Person.id} < \text{numtotalUser}$)
- The *creationDate* is a uniform random date between the generation parameters *startYear* and *endYear*.
- The first name of a person (property *firstName*) is correlated with attributes *locatedIn* and *gender*.

For each location (country) we have a set of names obtained from the *givennameByCountryBirthPlace* dictionary. First names are sorted by the name's popularity: $N1, N2, N3, N4, N5, N6$, where $N1, N3, N5$ are first names for male, and the others for female. The table below shows the popularity in increasing order per year period.

Time Period	Male Names	Female Names
1980 - 1985	[$N1, N3, N5$]	[$N2, N4, N6$]
1985 - 1990	[$N3, N1, N5$]	[$N4, N2, N6$]

- The last name (attribute *lastName*) of a person is *randomly* selected from the values available in the dictionary *surnameByCountryBirthPlace*. The procedure considers a correlation with the country of the person.
- The gender of a person (attribute *gender*) is generated randomly. The probability is 0.5 for selecting male or female as a value.
- The *birthday* of a person is a uniform random date between the 1/1/1980 and 1/1/1990.
- A person's email is obtained from dictionary *email*. Based on its popularity, each email domain is assigned with a probability. Such probability is used to select a domain for an specific person. There are some popular email domains such as Gmail. Top-5 email domains have the popularity scores. Others are randomly distributed.
- The value of attribute *speaks* is randomly generated but there is a correlation with the person's *location*. There is a chance to have English as a foreign language.
- *browserUsed* is randomly generated by selecting a browser from the dictionary *browsersDic*.
- The IP address for a user (attribute *locationIP* is correlated with the person's country. It uses the data specified in *ipzones* files.
- The value of the attribute *isLocatedIn* must be an instance of entity *City* (see Figure 3.1). The location identifier is set and also the Z-order of the location by using dictionary *dicLocation*. The value distribution follows the population of each country. First, a country is selected according to the probability of the population. Then, a random city is selected.
- The universities a person studies (*studyAt.name* values) are correlated with the location of the person (i.e., property *isLocatedIn*). Each location has a collection of universities (see dictionary *institutes-CityByCountry*). The top-10 universities will have much higher probability to be selected than the others (by default, it is 90%). The value of the attribute *classYear* for relation *studyAt* is given by the *birth year* of the person plus a random value between 20 and 25. The *city* of a university is determined by the *country* of the person (which is determined by his/her city). That is, the city of the university is located in the country of the person, but it does not imply that both cities are the same.

- The companies a person works for (*workAt.name* values) are correlated similarly to universities a person studies, with the location of the person. However, there is a small probability that both countries are not correlated.

The value of the attribute *workFrom* of relation *workAt* is given by the birth year of the person plus a random value between 20 and 25.

- The values for property *hasInterest* (i.e., tags) for a person *p* are generated as follows:
 1. Select a basic (main) tag from dictionary *dicCelebritiesByCountry*
 - (a) obtain the identifier of *p.isLocatedIn*
 - (b) use the dictionary *dicCelebritiesByCountry*, get a random celebrity related to the person's country with a 50% probability. If there are no celebrities for the country then select a random one.
 2. determine the number of tags by selecting a random value between parameters *maxNumTagsPerUser* and *maxNumTagsPerUser*.
 3. select additional tags
 - (a) get topics related to the main topic using dictionary *topicMatrixId*.
 - (b) select a random topic in the dictionary, if the main topic does not have any related topics.
- *likes*: The number of "likes" for a specific Post is defined as a random number between one and ten. The persons to be related to a specific post by a "like", are selected from the list of friends of the creator of the post.
- Relation *knows* has been designed to follow a power-law distribution by using the SSJ library [5]. The number of friends for a person is determined by function *PowerDist* of the SSJ library. The assigned friendships are divided in the following fashion: 45% will be correlated by university-city data, 45% from the tags, and the last 10% will be random. We have to note here that this is one of the most important relationships in a social network.
- *follows*: this feature is not yet implemented.

Entity Forum

Recall that each forum corresponds a user's (i.e., person's) wall.

- Internally, the generator considers three types of forums: *wall*, *photo album*, and *group*. For each person:
 1. the number of "wall forums" is given by the number of months between the person's *creationDate* and the parameter *endYear*.
 2. the number of "photo album forums" is a random number between 0 and 5.
 3. the number of "group forums" is a random number between 0 and 4.

The value of the identifier of a person's *p* forum (i.e., wall) is generated by multiplying *p.id* by 2. The other ones are generated sequentially after the last user wall id.

- There are three possible patterns for the value of property *title*: (i) for the user wall the title is: Wall of [firstname] [lastname] (ii) for the albums are: Album [number] of [firstname] [lastname] and (iii) for the groups are: Group for [celebrity] in [place]
- The value for a forum's *creationDate* is a random date between the person's *creationDate* and *endYear*.
- Relation *hasModerator* points to the person who is the creator of the forum. For each person a wall is created, for each month between the person's *creationDate* and *endYear*, a random number of album forums and of group forums are created.

- Attribute *hasTag* has as value a random subset of the tags that the person knows (through relation *knows.hasInterest*).
- Relation *hasMember* connects a forum to a subset of the persons that the forum's moderator knows (through relation *knows*).
- A forum is connected to a randomly generated number of posts through attribute *containerOf*. The number of posts connected to a forum is a random number between 1 and 20.

Entity Post

- A post's identifier (*id*) is an integer, assigned in a sequential manner to a post.
- For each user and for each month a certain amount of posts are generated as defined in the private parameter file. The user and the generated posts are connected using the *hasCreator* attribute.
- The *creationDate* of a post is a random date between the post's creator *creationDate* (path *hasCreator.creationDate*) and *endYear*.
- The value of attribute *browserUsed* is the browser used by the creator of the post (path chain *hasCreator.browserUsed*).
- Attribute *locationIP* is correlated to the creator's country but there are certain chances to have a randomIP in the summer season and another in normal season.
- The content of a post (attribute *content*) is a fraction from the abstracts of the tags available in dictionary *tagText*.
- The language of the post (attribute *language*) exists only for text posts. It is selected randomly from the set of languages the creator of the post speaks (path *hasCreator.speaks*).
- The image file of a post (attribute *imageFile*) exists only for image posts. The value of the attribute is a text with pattern "photo[number].jpg"
- The value for attribute *isLocatedIn* is the country associated with the post's *locationIP*.
- The values for attribute *hasTag* are selected from the list of user's tags (through path *hasCreator.hasInterest*). Each tag has a 1/5 chance of being selected except one selected to force a non empty set. The number of tags for a post is in the range $[1, |T|]$ where T is the number of user's tags.

Entity Comment

- A comment's identifier *id* is an integer, assigned in a sequential manner to a comment.
- A random friend or member of the forum is selected as the creator of the comment (attribute *hasCreator*).
- The *creationDate* of a comment is a random date between the last comment of the base post *creationDate* (path *replyOf.creationDate*) and *replyOf.creationDate* + one day.
- The value of attribute *browserUsed* is the browser used by the creator of the comment (path chain *hasCreator.browserUsed*).
- Attribute *locationIP* is correlated to the creator's country but there are certain chances to have a randomIP in the summer season and another in normal season.
- The content of a comment (attribute *content*) is a fraction from the abstracts of the tags available in dictionary *tagText*.

- The value for attribute *isLocatedIn* is the country associated with the comment's *locationIP*.
- There is a probability of $\frac{1}{|N| + 1}$ where N is the number of created comments, to be replies of the original post (attribute *replyOf* to a post).
- In a similar manner, there is a probability of $\frac{N}{|N| + 1}$ where N is the number of of created comments, to be replies of the original comment (attribute *replyOf* to a comment).

Entity Organization

- An organization's identifier *id* is an integer, assigned in a sequential manner to the organization.
- The type of an organization (attribute *type*) can be either a university or company. Organizations (universities and companies) are generated according to the generation of persons. Specifically, to allow correlations: *Person-isLocatedIn-City*, *Person-studyAt-University-City*, *Person- workAt-Company-Country*.
- The names for universities and companies are obtained from the dictionaries *institutesCityByCountry* and *CompaniesByCountry*.

Entity Tag

- A tag's identifier *id* is an integer, assigned in a sequential manner to a tag.
- Tag names are obtained from the *dicTag* dictionary.
- Tags are associated to tag classes (attribute *hasType*) through the *dicTag.txt* dictionary.

Entity TagClass

- A tag-class has an integer identifier *id*, assigned to it in a sequential manner.
- Tag-class names are obtained from the *tagClasses* dictionary.
- The taxonomies of tag-classes are obtained from the *tagHierarchy* and are captured in the *isSub-ClassOf* attribute.

Entity Place

- A place has an identifier *id* that is an integer, assigned in a sequential manner to it.
- There are three types of places: city, country and continent. The countries, continents and its hierarchy are extracted from dictionary *dicLocation.txt*. The cities and its hierarchy with countries are extracted from *institutesCityByCountry.txt*. This extraction is done in the start-up process of the generator creating a set of countries and cities available to be assigned. However not all of them may be used in the generation process.
- The names of continents, countries and cities are obtained from the *dicLocation* for the first two and from the *popularPlacesByCountry* dictionary for the last (values for attribute *name*).
- The relationship *isPartOf* between two places is obtained from the *dicLocation* and *popularPlaces-ByCountry* dictionaries. It defines a *hierarchy* of places (i.e, London is part of UK, UK is part of Europe) and is obtained by DBPedia.

3.4 Graph Overview

In Section 2.1.1 of LDBC Deliverable D3.3.1 [1] we summarized some properties that often appear in real graphs, and that will be useful to characterize them. Table 3.1 shows a comparison of the expected values for some of these metrics in a real social network, and the measured metrics of a socialnet_dbgen synthetic dataset describing the activities of 100k users during 1 year. In this case, the analysis has been done for *person-[knows]->person*, which is the most important relationship in a social network graph.

Table 3.1: person-[knows]->person sf=100K1Y statistics

Metric	Description	Expected	socialnet_dbgen
community structure	a very large connected component	at least 80% to 90% of the nodes	99.78%
small world property	average path length	smaller than 5 or 6	3.93
degree of transivity of the graph	average clustering coefficient	greater than 0.3	0.11
diameter	largest distance between two nodes	up to 9 or 11	11

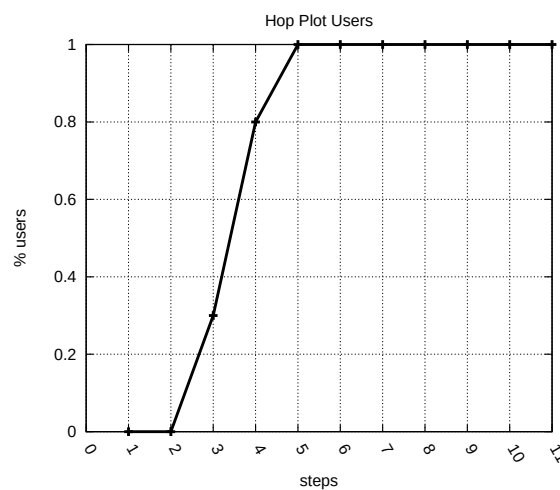
There are also two important diagrams that characterize the *knows* relationship: the *hop plot*, a visualization of the distribution of pairwise distances that grows between 2 and 5 as expected; and the *edge degree distribution* that clearly shows a power-law distribution, almost linear in a log-log diagram ⁵.

Table 3.2: Global Statistics

N	80,767,146	number of nodes
E	350,352,746	number of edges
V	500,108,979	number of attribute values

Table 3.3: User-Knows Subgraph Statistics

N	100,000	number of nodes
E	2,887,796	number of edges
N_c	99,778	number of nodes in largest connected component
N_c/N	1.00	fraction of nodes in largest connected component
C	0.11	average clustering coefficient
D	6	diameter
$\bar{D}$	3.93	average path length



⁵Note that the x-axis has been computed with $\log_{10}(\text{degree} + 1)$ to represent in a logarithmic scale all the range of degree values

Table 3.4: Labels

(a) Node

comment	61,832,051
emailaddress	171,049
forum	1,244,419
language	86
organisation	6,403
person	100,000
place	5,129
post	17,395,796
tag	12,143
tagclass	70

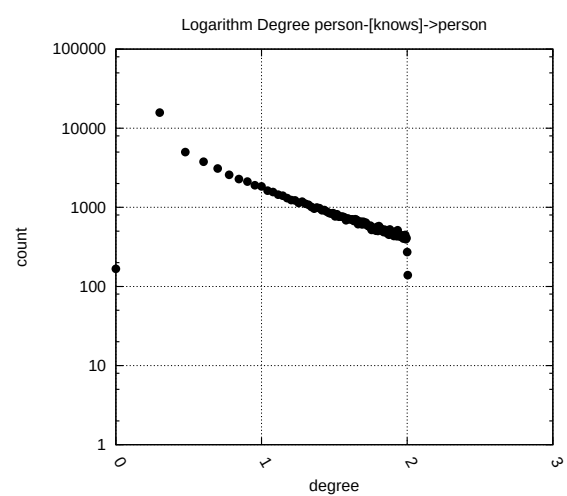
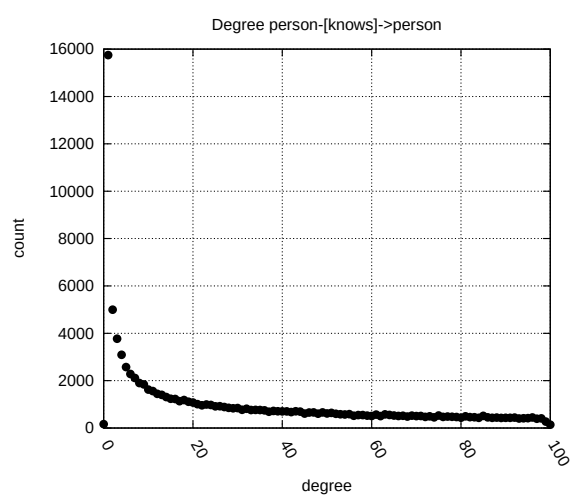
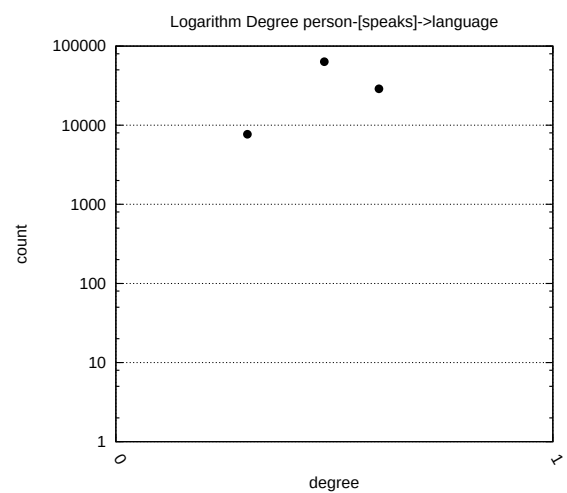
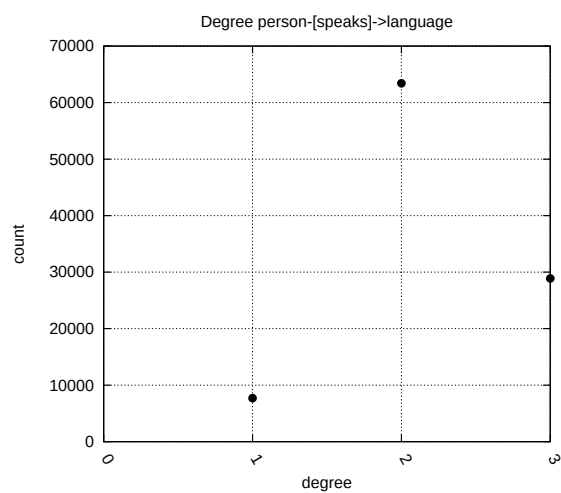
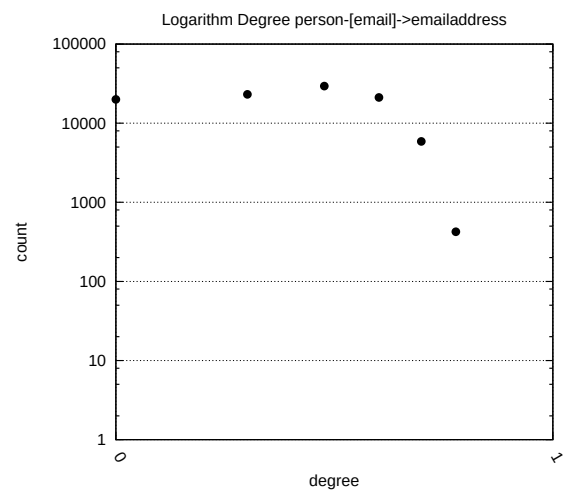
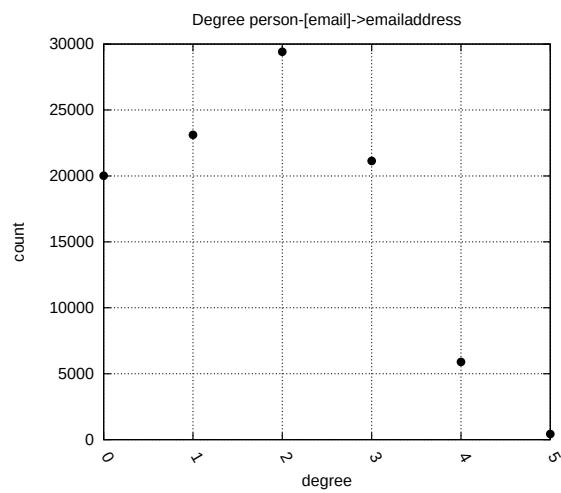
(b) Edge

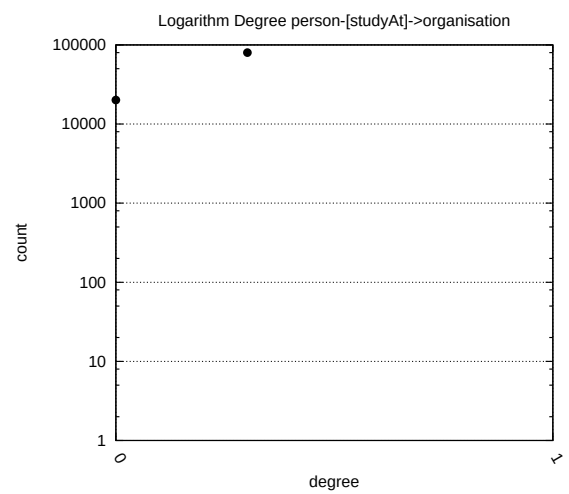
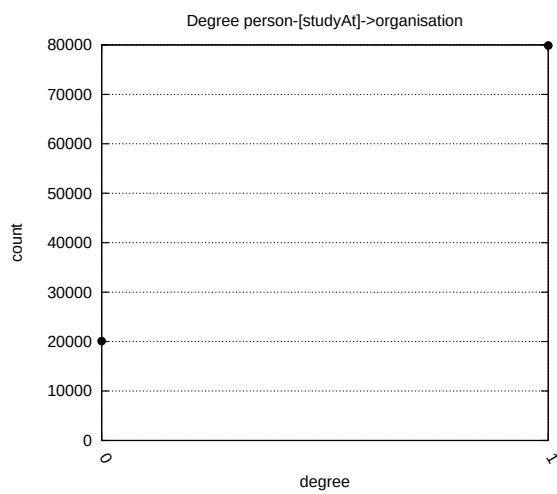
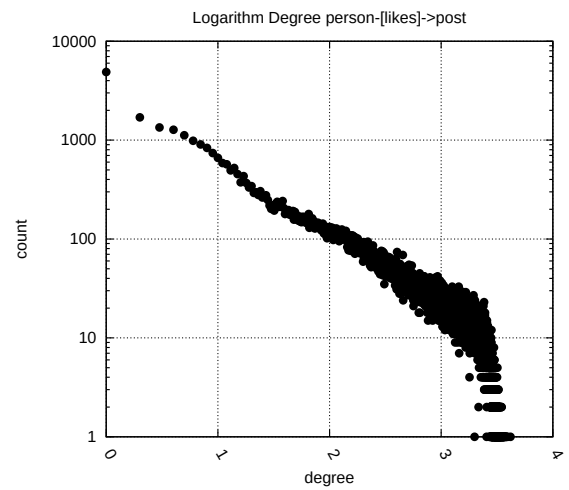
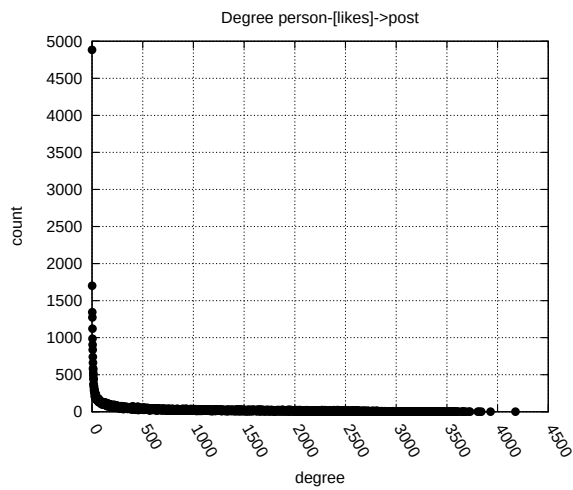
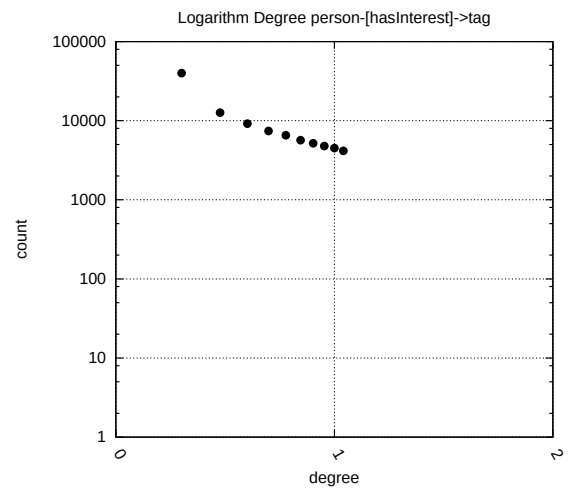
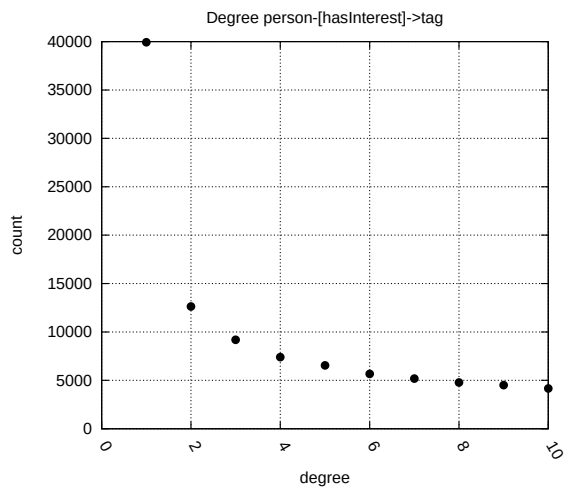
containerOf	17,395,796
hasCreator	79,227,847
hasInterest	345,807
hasMember	26,185,167
hasModerator	1,244,419
hasTag	15,045,764
hasType	12,143
isLocatedIn	79,334,250
isPartOf	5,123
isSubclassOf	69
knows	2,887,796
likes	66,142,152
replyOf	61,832,051
studyAt	79,894
workAt	222,222

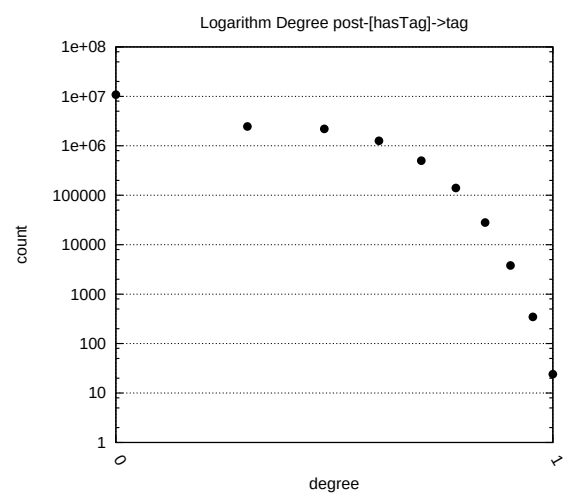
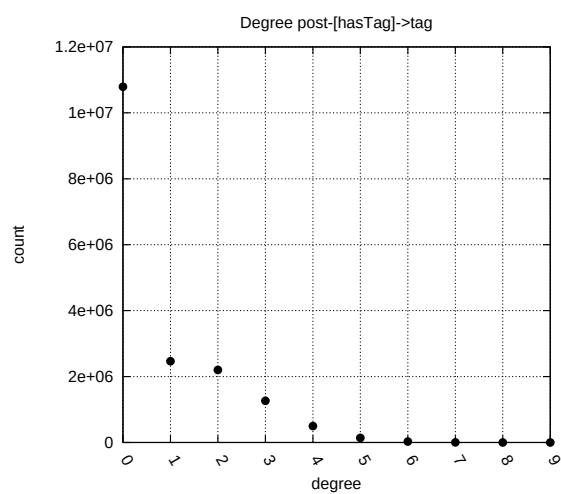
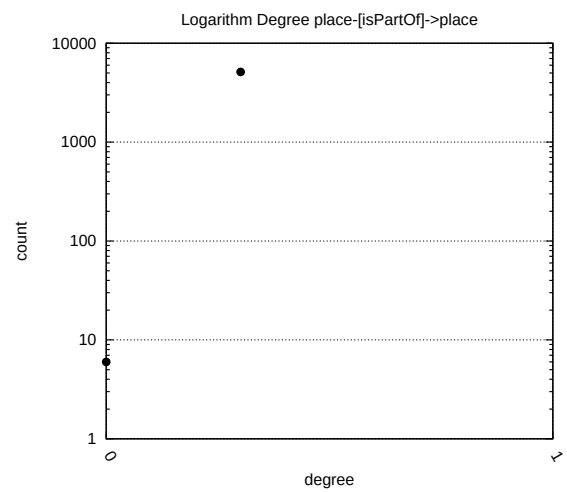
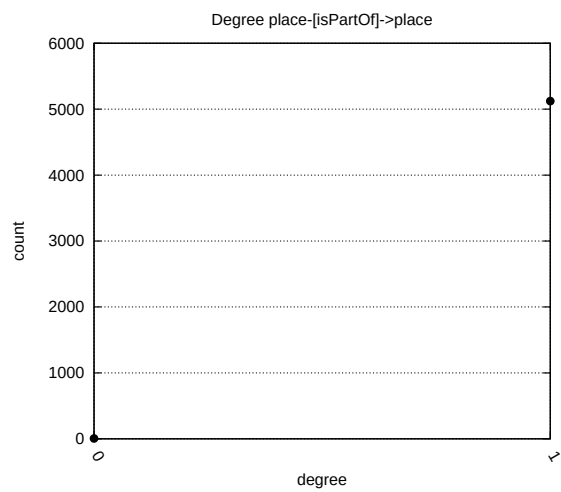
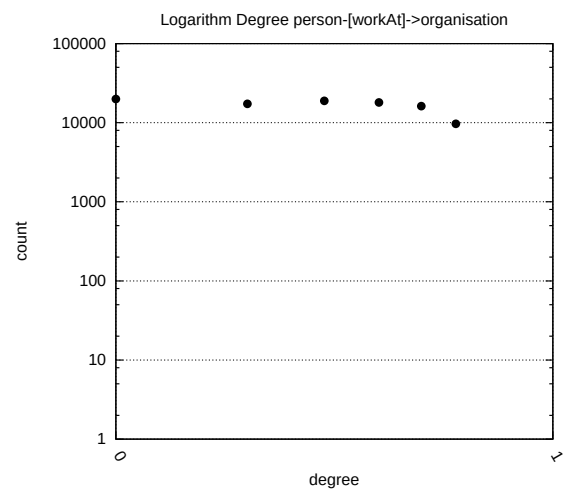
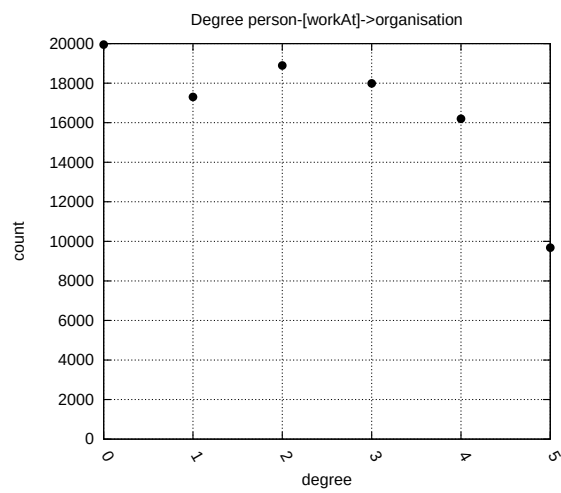
Table 3.5: Attributes

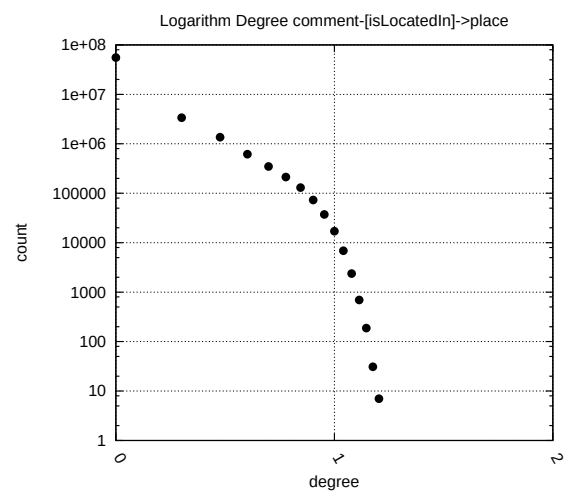
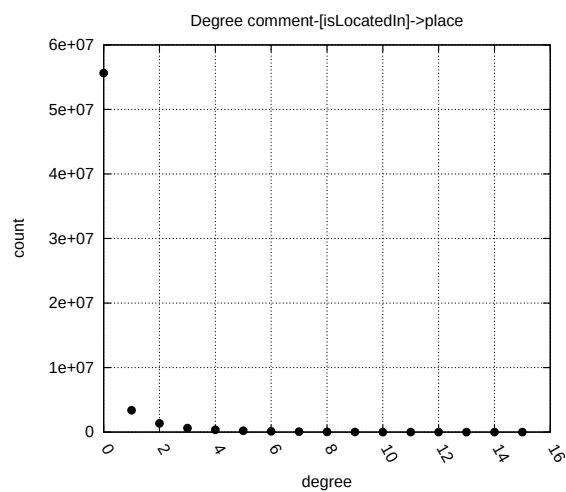
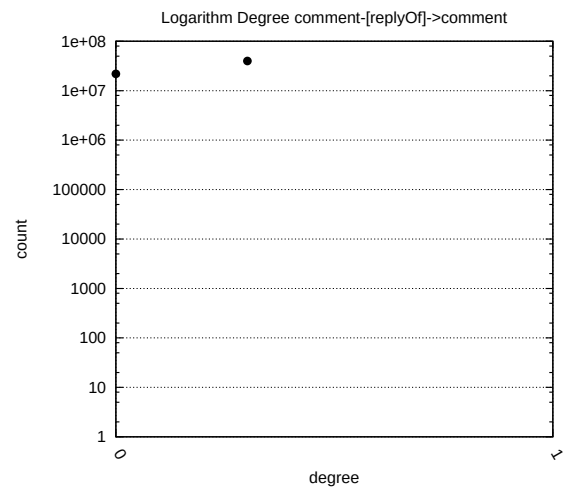
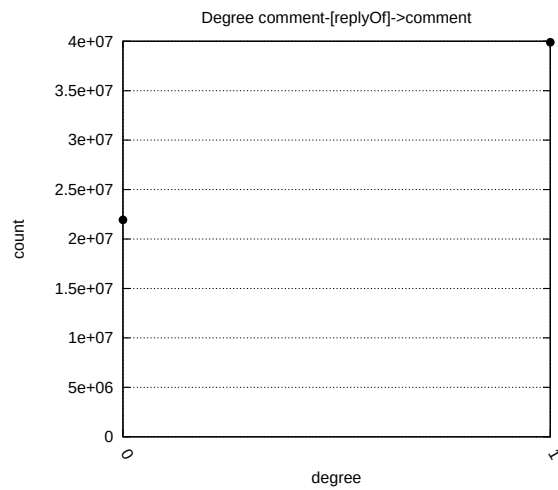
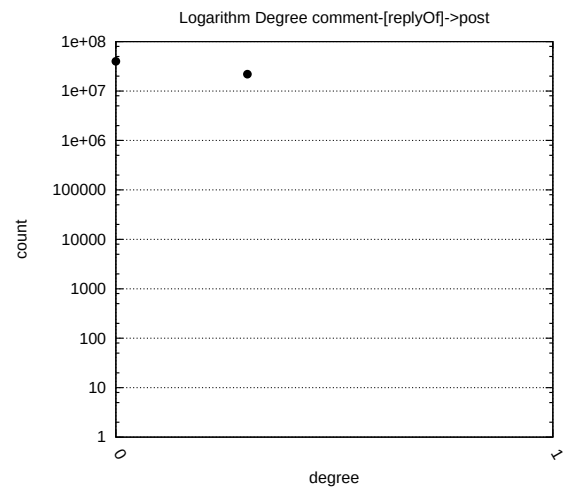
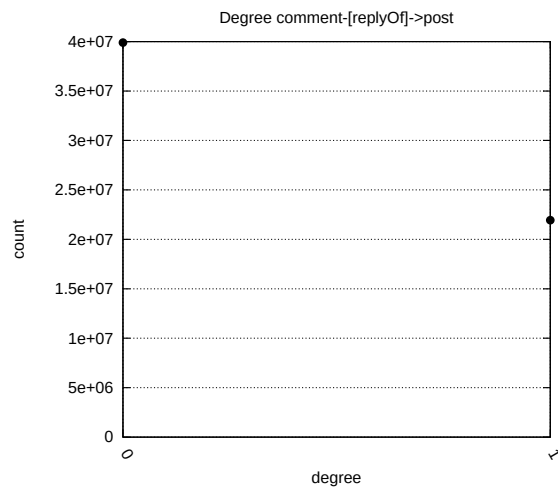
Type	Attribute	DataType	Count	# NULL	# Distinct	Lenght		
						Min	Max	Avg
comment	browserUsed	string	61,832,051	0	-	5	17	8.9
	content	string	61,832,051	0	-	21	443	166.7
	creationDate	timestamp	61,832,051	0	13,924,728	-	-	-
	id	long	61,832,051	0	61,832,051	-	-	-
	locationIP	string	61,832,051	0	-	7	15	10.8
emailaddress	email	string	171,049	0	171,049	13	55	20.1
forum	creationDate	timestamp	1,244,419	0	1,184,027	-	-	-
	id	long	1,244,419	0	1,244,419	-	-	-
	title	string	1,244,419	0	-	13	80	23.0
language	language	string	86	0	86	2	2	2.0
organisation	id	long	6,403	0	6,403	-	-	-
	name	string	6,403	0	-	3	98	27.2
	type	string	6,403	0	-	7	10	9.3
	url	string	6,403	0	-	31	126	55.2
person	birthday	timestamp	100,000	0	3,653	-	-	-
	browserUsed	string	100,000	0	-	5	17	9.0
	creationDate	timestamp	100,000	0	99,691	-	-	-
	firstName	string	100,000	0	-	1	36	5.4
	gender	string	100,000	0	-	4	6	5.0
	id	long	100,000	0	100,000	-	-	-
	lastName	string	100,000	0	-	2	19	5.3
	locationIP	string	100,000	0	-	7	15	10.8
place	id	long	5,129	0	5,129	-	-	-
	name	string	5,129	0	-	2	90	9.1
	type	string	5,129	0	-	4	9	4.1
	url	string	5,129	0	-	30	118	37.1
post	browserUsed	string	17,395,796	0	-	5	17	9.0
	content	string	6,602,448	10,793,348	-	21	442	166.5
	creationDate	timestamp	17,395,796	0	9,438,297	-	-	-
	id	long	17,395,796	0	17,395,796	-	-	-
	imageFile	string	10,793,348	6,602,448	-	11	17	16.4
	language	string	6,553,150	10,842,646	-	2	2	2.0
	locationIP	string	17,395,796	0	-	7	15	10.8
tag	id	long	12,143	0	12,143	-	-	-
	name	string	12,143	0	-	2	81	16.5
	url	string	12,143	0	-	30	108	44.5
tagclass	id	long	70	0	70	-	-	-
	name	string	70	0	-	4	26	10.4
	url	string	70	0	-	32	54	38.4
hasMember	joinDate	timestamp	26,185,167	0	7,215,600	-	-	-
likes	creationDate	timestamp	66,142,152	0	14,062,543	-	-	-
studyAt	classYear	string	79,894	0	-	4	4	4.0
workAt	workFrom	string	222,222	0	-	4	4	4.0

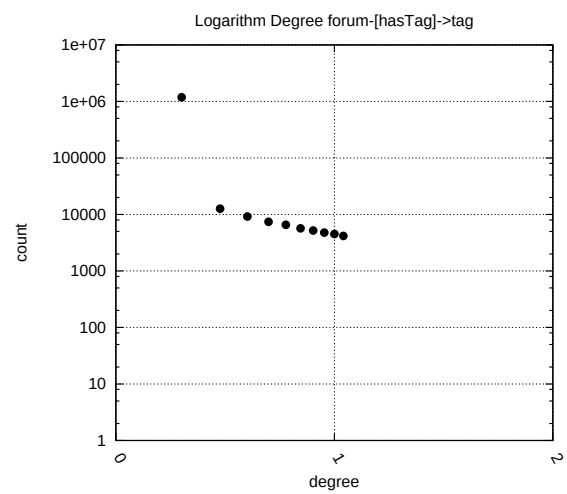
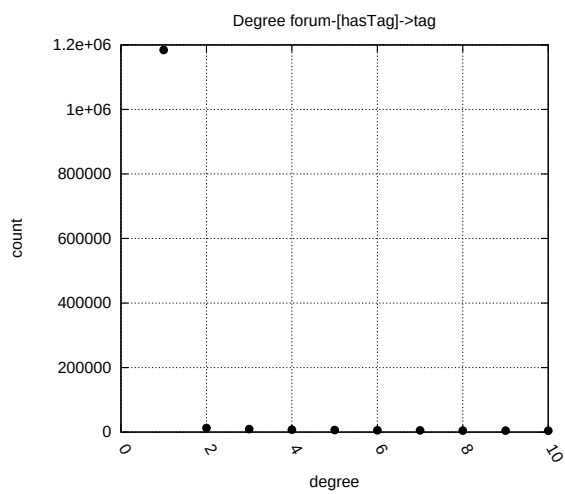
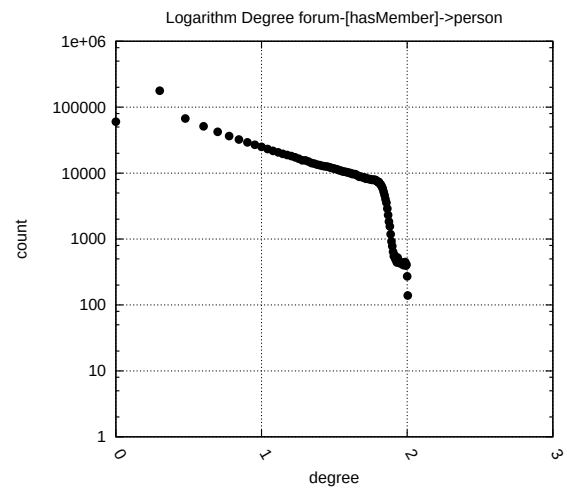
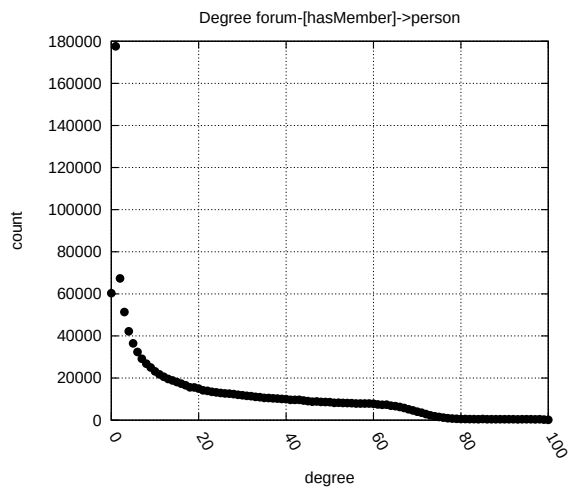
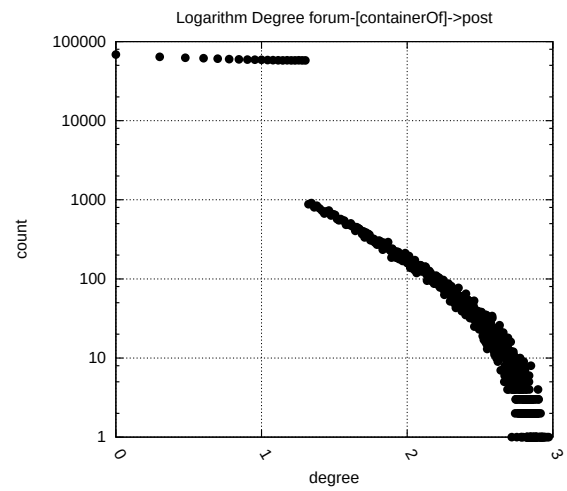
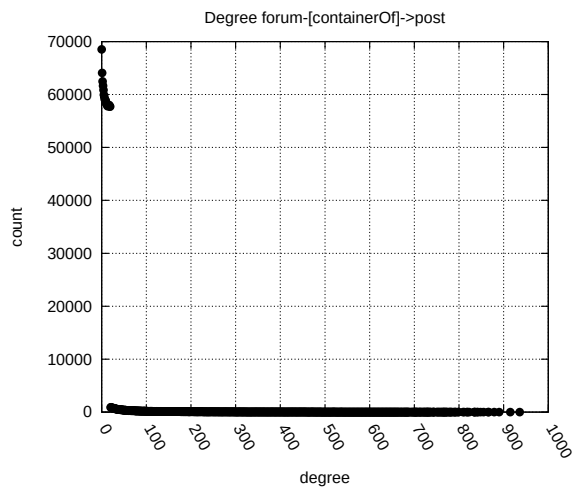
3.5 Degree distribution

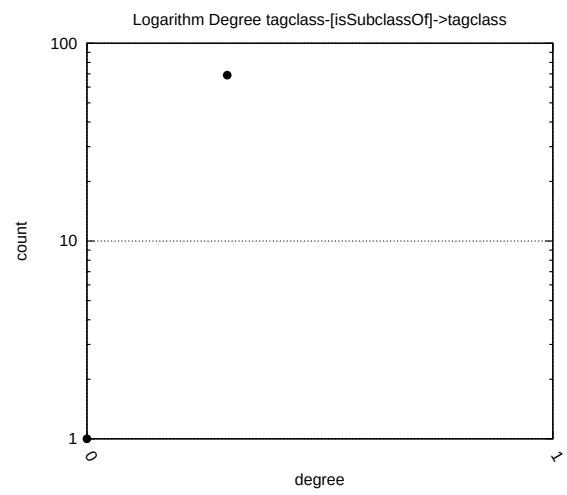
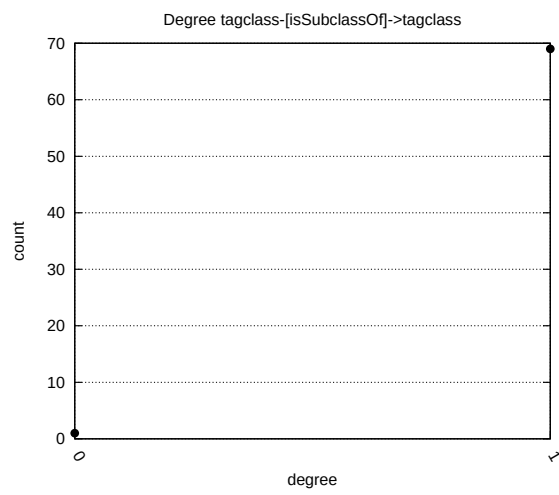




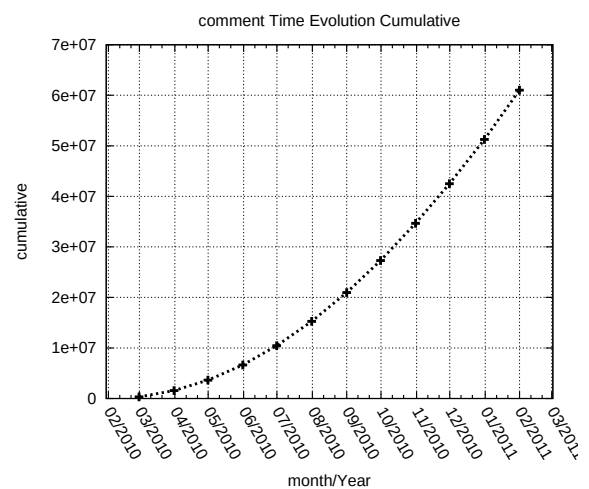
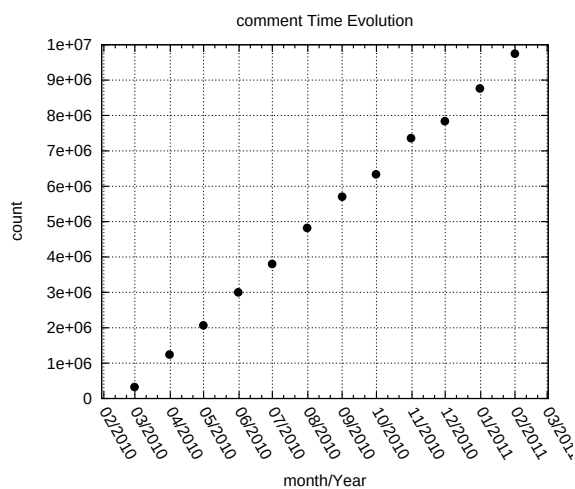
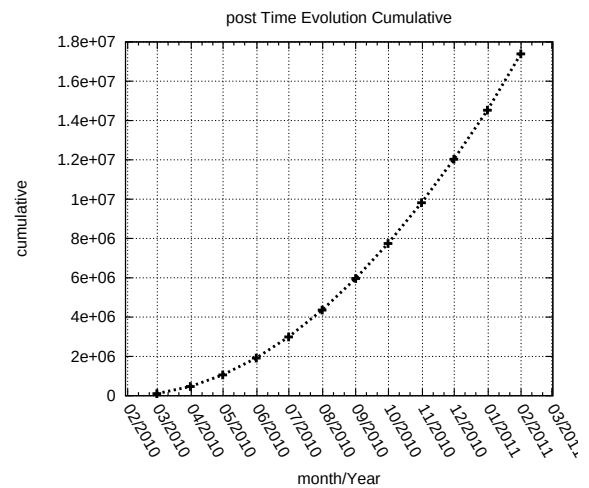
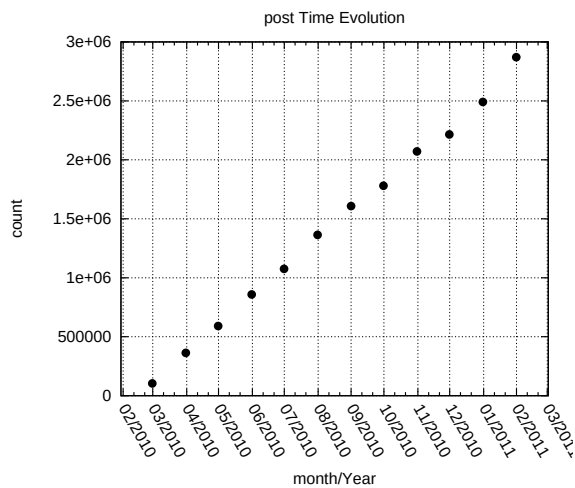
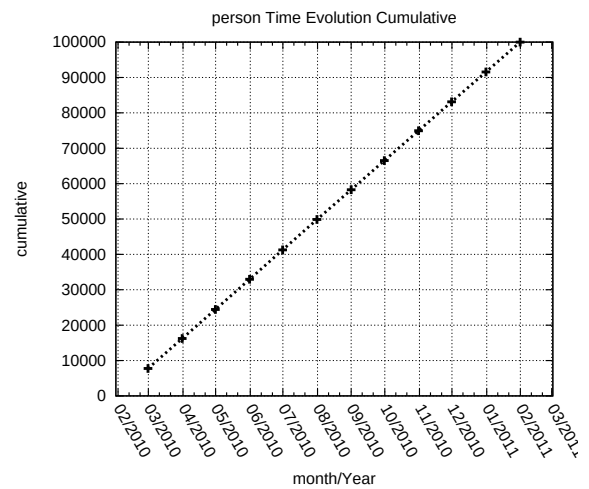
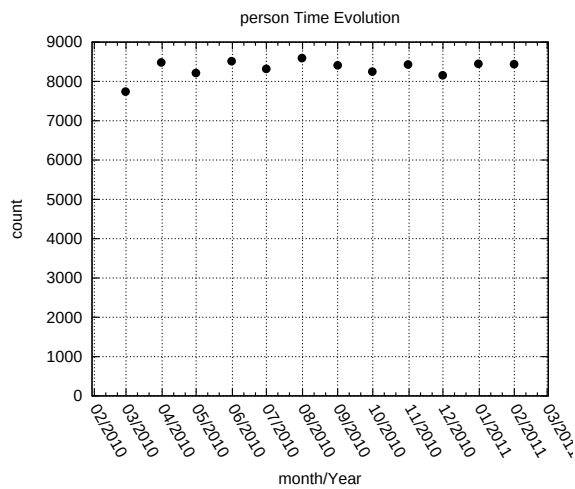


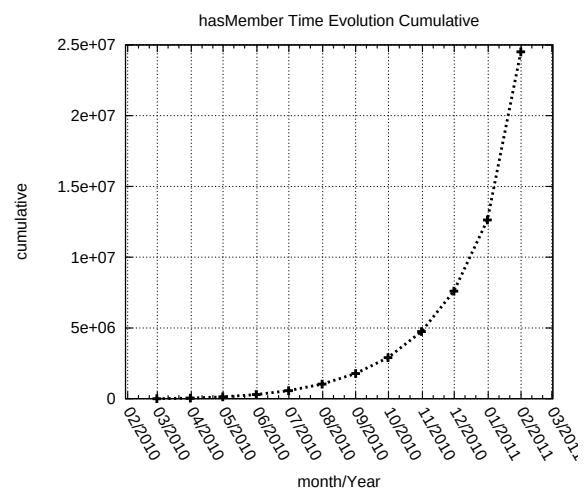
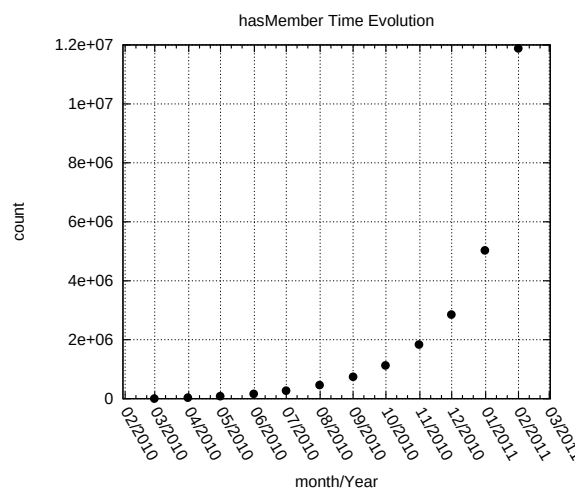
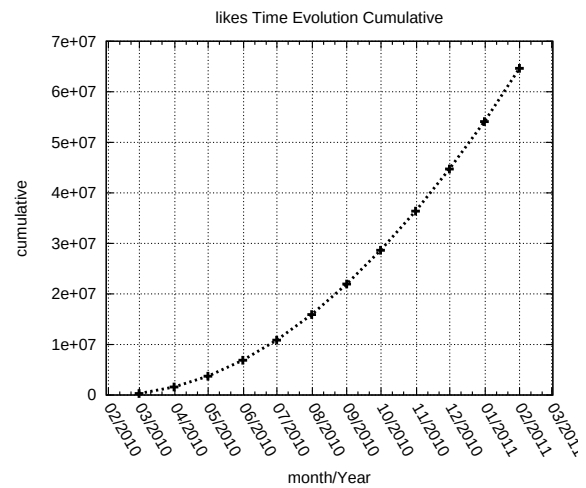
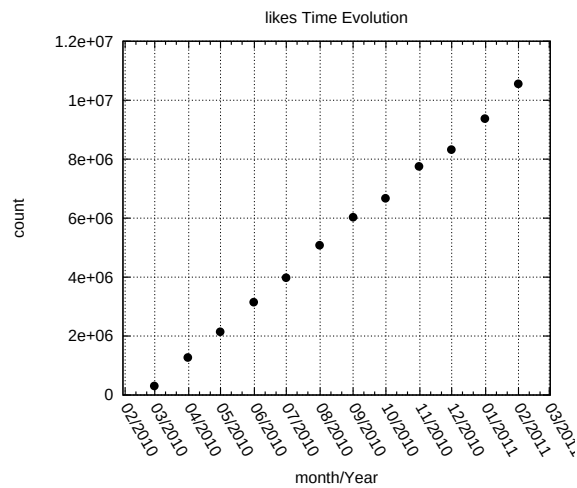
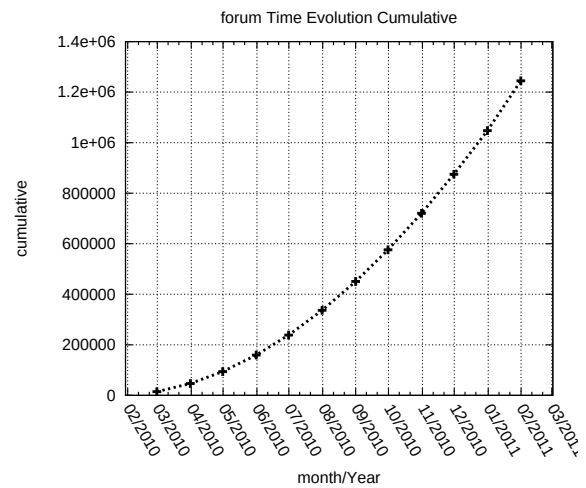
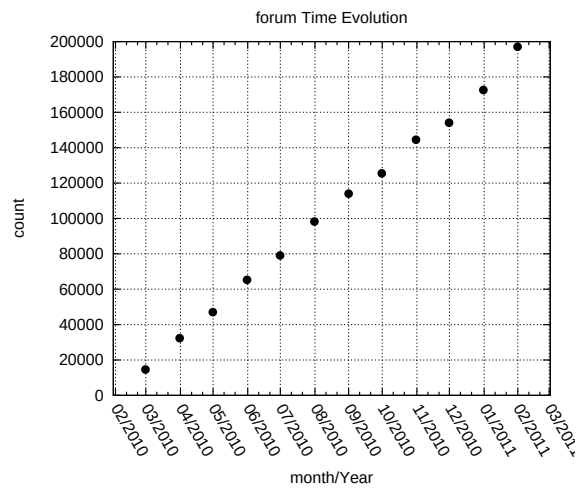






3.6 Time Evolution





4 CONCLUSIONS

In this deliverable we presented the data generators that will be used for developing the benchmarks associated with the two LDBC task forces, namely the *Semantic Publishing Task Force* and the *Social Network Task Force*. The data generators form one of the pillars of the benchmark design that will appear in future deliverables of this project.

Regarding the Semantic Publishing Benchmark, our effort was to build datasets that simulate real-world scenarios of this domain. To do so, the generator was based on statistics obtained by media organizations, in particular the BBC. The focus was to produce RDF datasets that model RDF descriptions of creative works. The data generator was designed to allow the generation of RDF descriptions of creative works and their associated tags in a way that the actual statistical distributions of the input reference dataset (BBC) are respected in the generated (artificial) datasets. The generator used the *editorial workflows* that are used by BBC to produce creative works.

The data generator for the Social Network Task Force can be used to create artificial graph databases that model a *social network*. The generator produces persons, information on persons, their network information and, most importantly, relationships between persons (representing the fact that a person knows another). The generator extended the S3G2 data generator [2] that targeted at modeling huge correlated directed labeled graphs. S3G2 and hence SNDG, use a *graph model* that is essentially a *correlated property graph*. Special care was taken in order for the generated data to be as realistic as possible using *correlations* between the different properties. More importantly, the relationships between persons were created in such a manner that important (and reasonable) correlations between the persons' information (e.g., nationality) and their relationship are created; for example, two people of the same nationality are more likely to know each other than two people of different nationality. The end result is a realistic simulation of real-world data, which can be reliably used for benchmarking purposes.

REFERENCES

- [1] Alex Averbuch. D3.3.1: Use case analysis and choke point analysis. LDBC Deliverable D3.3.1, 2013.
- [2] M-D. Pham, P.A. Boncz, and O. Erling. S3G2: a Scalable Structure-correlated Social Graph Generator. In *TPCTC*, 2012.
- [3] Y. Raimond, T. Scott, S. Oliver, P. Sinclair, and M. Smethurst. Use of semantic web technologies on the bbc web sites. http://3roundstones.com/led_book/led-raimond-et-al.html.
- [4] Geonames. <http://www.geonames.org/>.
- [5] SSJ: Stochastic Simulation in Java. <http://www.iro.umontreal.ca/~simardr/ssj/indexe.html>. University of Montreal.